GO 2 OF 3 · COMPONENTS AND INTERACTIONS

Open the boxes

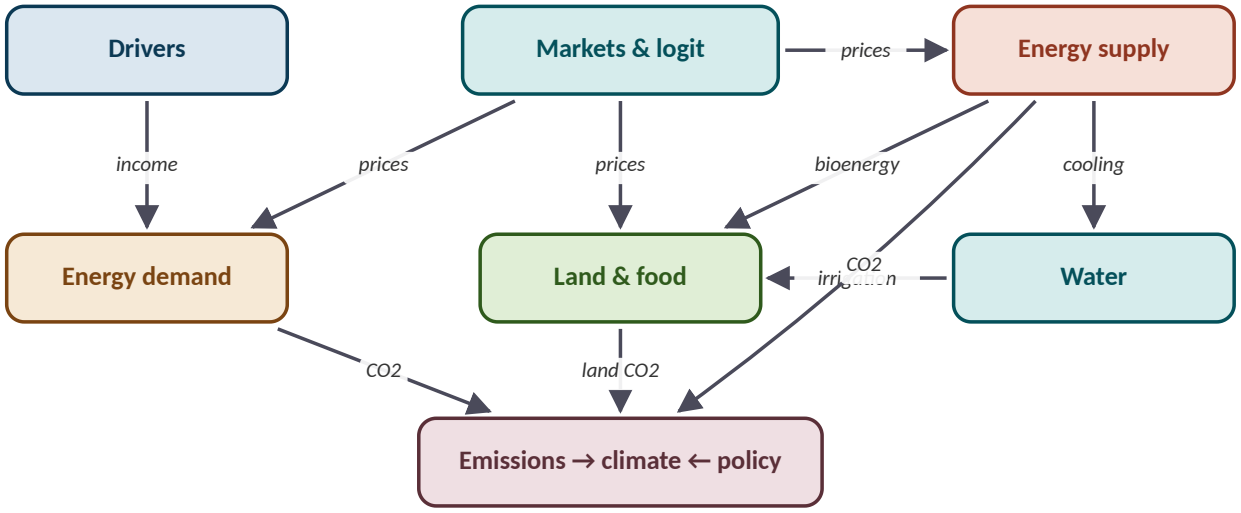
How each part of an integrated assessment model works, how the parts push on each other, and how to build a toy of each one.



INSIDE

- Eight components of GCAM, each with its rule, numbers and doc citations
- Interaction maps and six traced feedback loops
- Nine toy models in R and Python you run in minutes
- Where to see each mechanism in a real GCAM run

The map of Go 2



The Go 2 map. Eight boxes you will open, and the main arrows between them. Each module ends with its own interaction map; module 2.8 joins them.

HOW TO USE THIS HANDBOOK

1. Read a module's opening and mind map (5 min).
2. Work the SIMPLE CALCULATION by hand before reading on.
3. Run the BUILD IT script in R or Python; predict before every change.
4. Find the same mechanism in a real GCAM run (IN GCAM box).
5. Picture the factory stop; do TRY IT NOW; read the recap.

YOU WILL BE ABLE TO

- Describe each GCAM component, its main equation and its units.
- Draw who sends what to whom.
- Build a 20–40 line toy of each component in R or Python.
- Read the same mechanism in GCAM output with rgcam or gcamreader.
- Trace five cross-component feedback loops.

Modules: 2.1 Drivers · 2.2 Markets · 2.3 Logit · 2.4 Supply · 2.5 Demand · 2.6 Land · 2.7 Water · 2.8 System

Module	Kit script (python/ and R/)	Course lab	GCAM query (go2_queries.xml)
2.1 Drivers	m21_drivers	labs 3, 6	CO2 emissions by region
2.2 Markets	m22_markets	lab 4	CO2 prices
2.3 Logit	m23_logit	lab 7	elec gen by subsector
2.4 Supply	m24_supply	lab 8	elec gen by subsector
2.5 Demand	m25_demand	(Go 3 transport)	primary energy consumption by region
2.6 Land	m26_land	lab 9	aggregated land allocation
2.7 Water	m27_water	lab 9	water withdrawals by basin / region
2.8 System	m28a_climate, m28_system	labs 2, 10, 11, 12	global mean temperature; CO2 emissions

Before you start

You need	How to get it	Check it works
Python 3.10+ with numpy, scipy, matplotlib	`pip install numpy scipy matplotlib`	`python m23_logit.py` prints coal 0.539 at γ = −4
or R 4.x (base R is enough)	cran.r-project.org; RStudio optional	`Rscript m23_logit.R` prints the same shares
GCAM output for the IN GCAM boxes	your instructor's database folder, or run GCAM v9.1 yourself (8 GB RAM, Java)	`listScenariosInDB(conn)` shows Reference
rgcam or gcamreader	`remotes::install_github('JGCRI/rgcam')` or `pip install gcamreader`	`gcam/read_gcam.R` or `.py` writes 8 CSV files

Box	What it is for
HOOK	a surprising true fact to start the module
SIMPLE CALCULATION	the core rule worked by hand, every step shown
BUILD IT	the toy model: key lines, exact output, two predict-then-change tasks
IN GCAM	where the same mechanism lives in a real run: queries, files, labs
TRAP	the misconception that costs marks
PICTURE THIS	one stop on the factory tour, a memory image
TRY IT NOW	a 60–90 second exercise; answers at the back
QUESTIONS TO ASK OF ANY RUN	what a careful analyst checks before believing a result
RECAP SO FAR	every module so far in one line each

VERSION NOTE

The GCAM documentation (toc) describes **v8.2**; the latest release is **v9.1** (2 June 2026). Structure is taught from the v8.2 pages, and newer changes (KLEAM macro module, updated MAC curves, small modular reactors) are flagged as “latest”. Base year: 2021 (the overview page still says 2015).

THE GO 2 KIT

Folder `go2-kit/`: `python/` and `R/` hold the same nine toy models (m21–m28a, standard library + numpy/scipy in Python, base R in R); `gcam/` holds the query file and reader scripts for real GCAM output. See `go2-kit/README.md`.

MODULE 1 OF 8

2.1 Socioeconomic drivers

Everything an IAM computes starts from how many people there are and how rich they get.

HOOK

In GCAM's SSP runs, the world of 2100 has **6.9 billion** people in SSP1 but **12.7 billion** in SSP3, and income per person differs almost **sevenfold** between SSP3 ($12,092) and SSP5 ($83,496) (ssp). Same model, same physics. The storyline about people moved the answer more than any technology choice.



What this component represents

The **drivers** are the size of the economy and the number of people, region by region and period by period. They are not results of the energy or land system: in GCAM's classic mode they are inputs. Every demand in the model scales with them: floor space, steel, passenger-kilometres, food and municipal water.

How GCAM represents it

GCAM has **32 geopolitical regions** and runs in **five-year steps from 2025 to 2100** after calibrating to history (overview, common_assumptions). The exogenous inputs are population, labour productivity growth, labour force participation and base-year GDP (economy). Population comes from the UN to 2015 and from the SSP database afterwards; future GDP comes from the SSP database (inputs_economy).

Since GCAM v7 a **macro module (KLEM)** can make GDP respond to energy: effective energy enters a nested production function, so a policy that makes energy dearer can lower GDP and consumption (economy). GCAM v9.0 extends this to **KLEAM**, adding agriculture (released June 2026; documented in a change proposal, not yet in the v8.2 pages).

$$\mathrm{GDP}_{r,t} = \mathrm{Population}_{r,t} \times (\mathrm{GDP\ per\ person})_{r,t} \tag{1}$$

$$\mathrm{CO_2} = \mathrm{P} \times (\mathrm{GDP/P}) \times (\mathrm{E/GDP}) \times (\mathrm{CO_2/E}) \quad (\text{Kaya identity}) \tag{2}$$

SIMPLE CALCULATION

A region of 50 million people earns $20,000 each. Its income per person grows 2% a year for 25 years. What is GDP in year 25?

1. GDP today = 50 million × $20,000 = $1,000 billion = $1.0 trillion.
2. Growth factor = 1.02^{25} = 1.641 (about 64% more income per person).
3. If population is flat, GDP in year 25 = 1.0 × 1.641 = $1.64 trillion.
4. If population also grows 10% over the period: 1.64 × 1.10 = $1.81 trillion.

Answer: $1.64 trillion with flat population; $1.81 trillion with 10% more people.

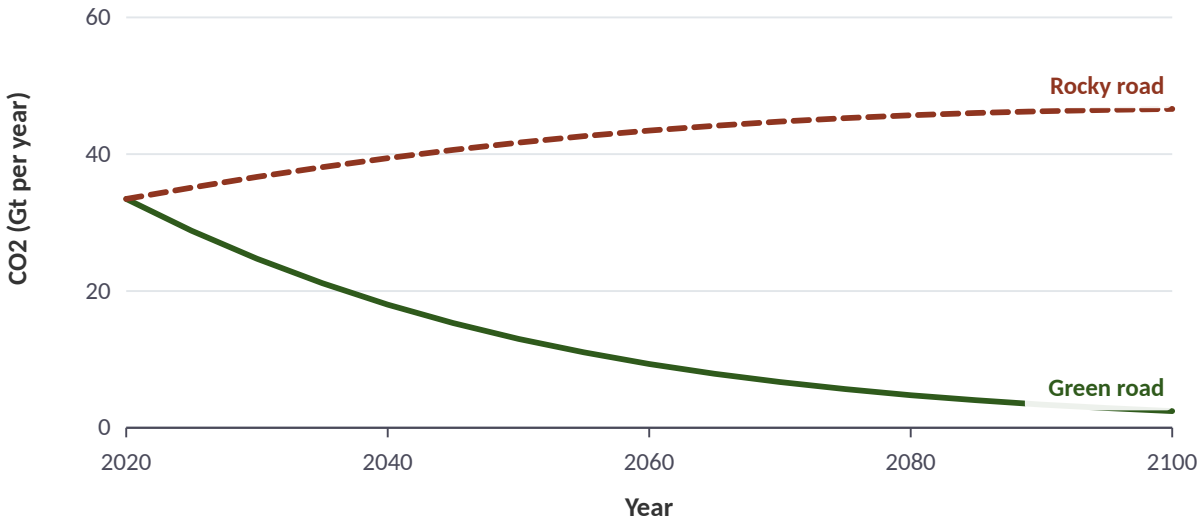


Figure 2.1 — two storylines, one formula. CO2 from the Kaya identity for two invented storylines in the spirit of SSP1 and SSP3. Notice the Green road falls to 2.4 Gt by 2100 even though income grows faster: energy and carbon intensity fall much faster. Illustrative numbers from `m21_drivers.py`.

SSP (GCAM, 2100)	Population	GDP per person	What drives it
SSP1 Sustainability	6.9 billion	$46,306	education, fast productivity
SSP2 Middle of the road	9 billion	$33,307	trends continue
SSP3 Regional rivalry	12.7 billion	$12,092	slow growth, high fertility
SSP5 Fossil-fuelled	7.4 billion	$83,496	fast growth, high demand

Values as listed on the GCAM SSP page (ssp); SSP4 is not listed there. GCAM was the marker model for SSP4.

Who it talks to

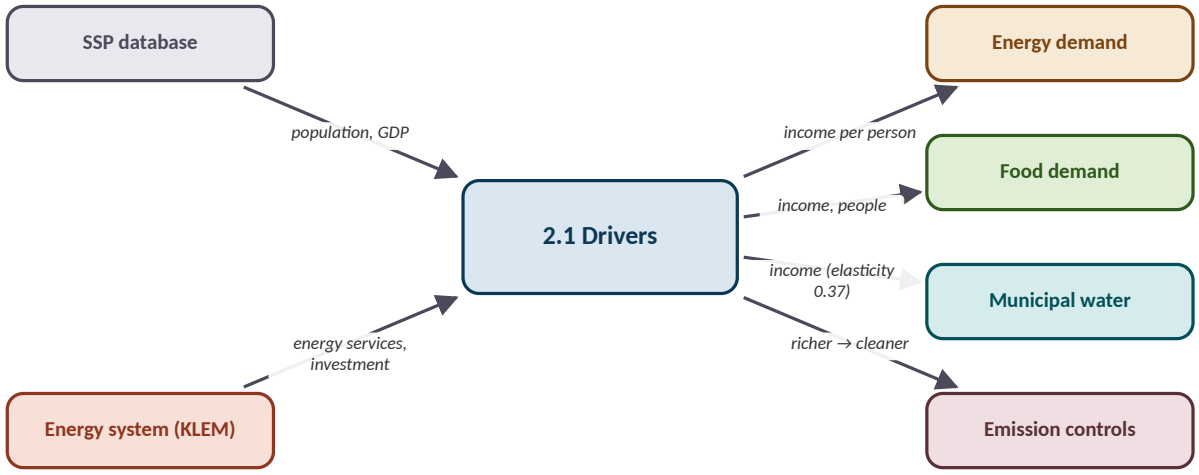


Figure 2.2 — interaction map. Drivers mostly *send*. Only in the macro mode do they also *receive* (from energy).

BUILD IT

Script: `python/m21_drivers.py` (same logic in `R/m21_drivers.R`). Key lines:

```
stories = {
    "Green road": dict(pop=population(7.8, 8.6, 0.05), inc=income(11, 0.022), ei=-0.025,
ci=-0.030),
    "Rocky road": dict(pop=population(7.8, 12.5, 0.03), inc=income(11, 0.010), ei=-0.008,
ci=-0.003),
}
```

It prints:

```
Green road: factor ratios 2100/2020 = [1.1   5.703 0.132 0.087], product = 0.072, CO2 ratio
= 0.072
Rocky road: factor ratios 2100/2020 = [1.52  2.217 0.526 0.786], product = 1.393, CO2 ratio
= 1.393
```

Change this and predict first:

1. Raise the Rocky road income growth from 1.0% to 2.0% a year. Predict: does CO2 in 2100 rise or fall, and by roughly how much? Then run.
2. In the Green road, set carbon-intensity change to 0. Which Kaya ratio changes, and does the product still equal the CO2 ratio?

IN GCAM

See the drivers in a real run: in ModelInterface open *socioeconomics → GDP by region* and *population by region* (standard query file `output/queries/Main_queries.xml`; user-guide). In R:

```
library(rgcam)
conn <- localDBConn("/course/gcam/output", "database_basexdb")
prj  <- addScenario(conn, "go2.dat", "Reference", "go2_queries.xml")
co2  <- getQuery(prj, "CO2 emissions by region")   # labs 7 and 10
```

To switch storyline, GCAM ships SSP configuration files; each changes population and GDP and a set of SSP-specific technology and land assumptions (ssp).

TRAP

“GDP is an *output* of GCAM, so the model predicts growth.” In the classic setup GDP is an **input**. Even with the macro module, the long-run path is calibrated to a chosen SSP: the model adjusts it, it does not discover it.

PICTURE THIS · STOP 1 OF THE ROUTE

Goods-in dock. Lorries back up to the factory gate. Each carries a crate stamped with a **population number** and a crate full of **banknotes**. A forklift multiplies them on a giant calculator bolted to the wall. Nothing in the factory moves until these crates are unloaded. *Mapping: drivers come in from outside; GDP = people × income.*

TRY IT NOW

(90 s) A region's population grows 20%, income per person 50%, energy per $ falls 30% and CO2 per unit of energy falls 20%. Without a calculator, will CO2 go up or down? Then check: 1.2 × 1.5 × 0.7 × 0.8.

QUESTIONS TO ASK OF ANY RUN

1. Which SSP drives the run, and is GDP fixed or responding through the macro module?
2. Are regional population and GDP harmonised to the scenario database you cite?
3. Does a 32-region aggregate hide the country you care about?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.

MODULE 2 OF 8

2.2 Markets, equilibrium and solving

GCAM is a giant auction: it hunts for the prices that make supply equal demand everywhere, at once.

HOOK

GCAM does not optimise the future. Each five-year period it solves “a few hundred” markets *simultaneously* and then steps forward, like a person who decides with today's information (solver, overview). That is why it can build a coal plant in 2030 that a policy strands in 2040.



What this component represents

A **market** is a place where a good is bought and sold at one price: crude oil, regional electricity, corn in a region, water in a basin, carbon if there is a policy. An **equilibrium** is a set of prices where, in every market, the amount demanded equals the amount supplied. GCAM's core job is to find that set of prices each period.

How GCAM represents it

“GCAM operates by determining a set of prices that ensure supply is equal to demand for all time steps” (marketplace). The solver treats this as finding the price vector *p* where excess demand *F(p)* = 0, working in **log prices** with scaled excess demands (solver).

$$\mathrm{F_m(p) = Demand_m(p) - Supply_m(p) = 0 \quad for\ every\ market\ m} \tag{3}$$

$$\mathrm{B \cdot \delta x = -F(x),\ \ x \leftarrow x + \lambda\, \delta x,\ \ \lambda \leq 1\ (Broyden\ step\ on\ log\ prices)} \tag{4}$$

Broyden's method is the main workhorse: it guesses how all excess demands react to all prices (an approximate Jacobian *B*) and updates that guess after each step instead of recomputing it. Bisection moves prices near the solution first; a preconditioner sets sensible starting prices. Newton–Raphson is deprecated (solver). The sample configuration uses a relative `solution-tolerance` of 0.001 and up to 2,500 model calculations per period (example values, not stated defaults).

SIMPLE CALCULATION

One market: demand = 1000 − 4p and supply = 6p. A $50/t carbon tax on a fuel with 0.9 t/MWh shifts supply to 6(p − 45). Find both prices.

1. No tax: 1000 − 4p = 6p → 1000 = 10p → p = 100; quantity = 600.
2. Tax per MWh = 0.9 × 50 = 45, so sellers keep p − 45.
3. With tax: 1000 − 4p = 6(p − 45) = 6p − 270 → 1270 = 10p → p = 127.
4. Quantity = 1000 − 4 × 127 = 492. Buyers pay 27 more; sellers keep 18 less (27 + 18 = 45).

Answer: p = 100 → 127; quantity 600 → 492.

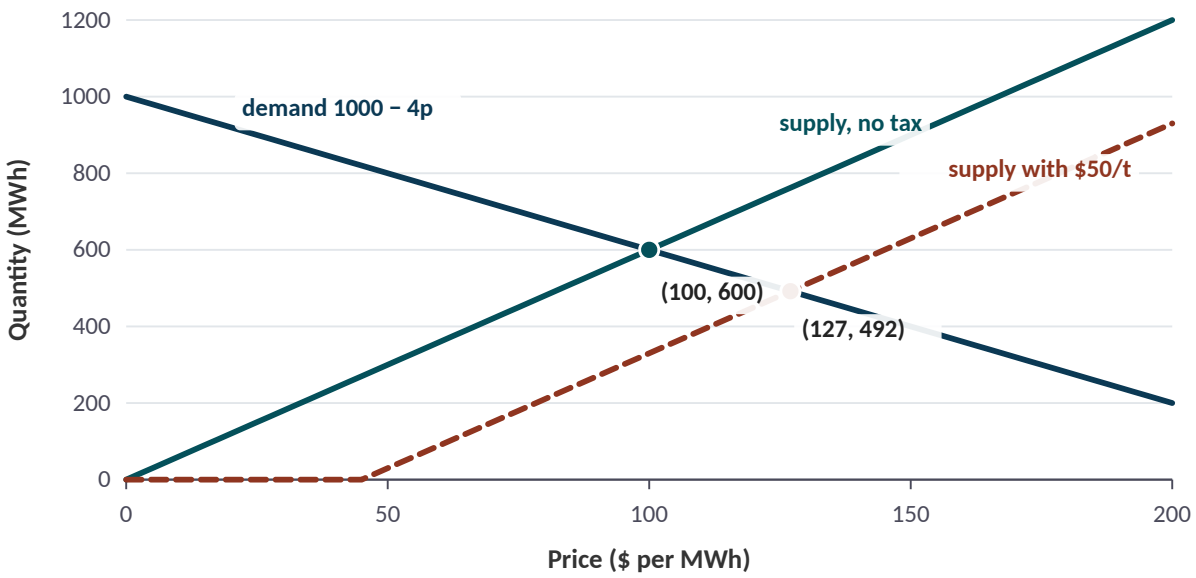


Figure 2.3 — one market, solved by hand. The tax shifts the supply curve right by 45 $/MWh of price. Notice the new crossing sits less than 45 higher: the burden is shared.

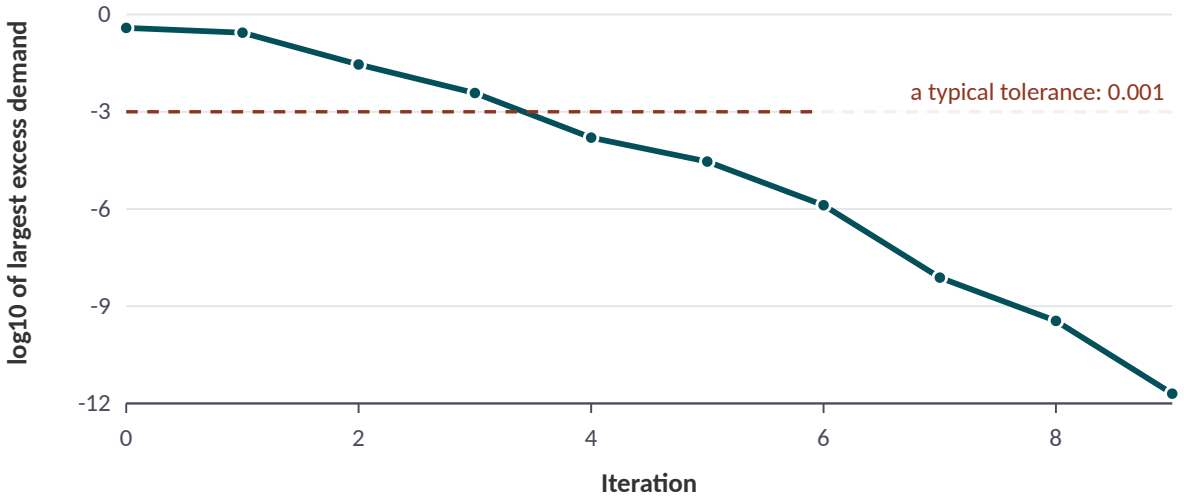


Figure 2.4 — three linked markets solved together. A course-written Broyden solver (with step halving, as GCAM does) on the oil–electricity–biomass toy with a $50 carbon price. It converged in 9 iterations to oil 9.04, electricity 31.60, biomass 5.25. Notice the error does not fall smoothly at first: the solver is still learning how the markets react to each other.

Who it talks to

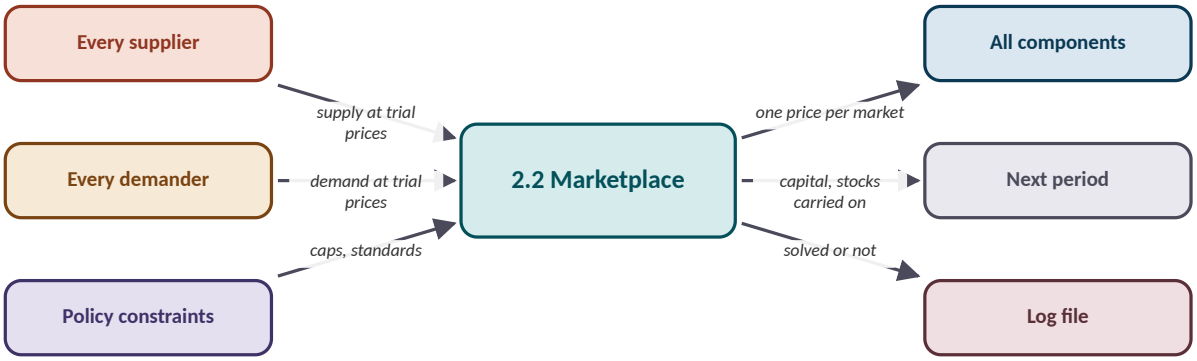


Figure 2.5 — interaction map. The marketplace is the switchboard: it hears every supply and demand and sends back one price per market.

BUILD IT

Script: `python/m22_markets.py` (same logic in `R/m22_markets.R`). Key lines:

```
def f(logp, carbon):
    p = np.exp(logp)
    return excess3(p, carbon) / np.array([100, 80, 20])

for carbon in (0, 50):
    sol = root(f, x0=np.log([8, 25, 6]), args=(carbon,), method="broyden1", options=
{"fatol": 1e-10})
    p = np.exp(sol.x)
    print(f"Three markets, carbon {carbon:>2}: prices oil {p[0]:.2f}, electricity
{p[1]:.2f}, "
          f"biomass {p[2]:.2f}; converged={sol.success}; max |excess| = {abs(excess3(p,
carbon)).max():.1e}")
```

It prints:

```
One market, tax  0: price  100.0, quantity  600.0
One market, tax 50: price  127.0, quantity  492.0
Three markets, carbon  0: prices oil 10.00, electricity 30.00, biomass 5.00; converged=True;
max |excess| = 1.6e-10
Three markets, carbon 50: prices oil 9.04, electricity 31.60, biomass 5.25; converged=True;
max |excess| = 1.8e-10
```

Change this and predict first:

1. Change the carbon price from 50 to 150. Predict the direction of each of the three prices before you run it. Why can the oil *producer* price fall?
2. Start the solver far away, at x0 = log([1, 100, 50]). Does it still converge? Count the iterations (print `sol.nit`).

IN GCAM

Every run writes `exe/logs/main_log.txt`; a good run ends with "Model exiting successfully" and each period reports whether it solved (user-guide). The solver settings live in a file named by `<Value name="solver_config">` in `configuration.xml`, with per-year `<user-configurable-solver>` blocks (solver). Market prices are reported by the query *prices of all markets*; market names join region and good, e.g. `USACorn` (outputs_prices). Lab 4 (`uniroot`) is the one-market version of the same search.

TRAP

"GCAM finds the *optimal* energy system." No: it finds **market-clearing prices** period by period with no foresight. Optimisation models (MESSAGEix, TIMES; module 2.8 contrasts them) minimise total cost over the whole horizon instead.

PICTURE THIS · STOP 2 OF THE ROUTE

Auction hall. A hall with hundreds of tiny auction podiums, one per market. A single auctioneer with a **log-scale megaphone** shouts all prices at once, watches the piles of unsold goods shrink, and only lets the factory clock tick forward five years when every pile is flat. *Mapping: all markets clear together; prices in logs; recursive steps.*

TRY IT NOW

(60 s) In the one-market example, change demand to 1000 − 2p. Solve for the price with no tax and with the $50 tax. Who carries more of the tax now, buyers or sellers?

QUESTIONS TO ASK OF ANY RUN

1. Did every period solve? Search `main_log.txt` for unsolved periods before trusting anything.
2. Which dollar year are prices in (1975$ for energy, 1990$ for carbon)?
3. Is a surprising result a real equilibrium effect or a solver artefact (prices at a bound)?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.

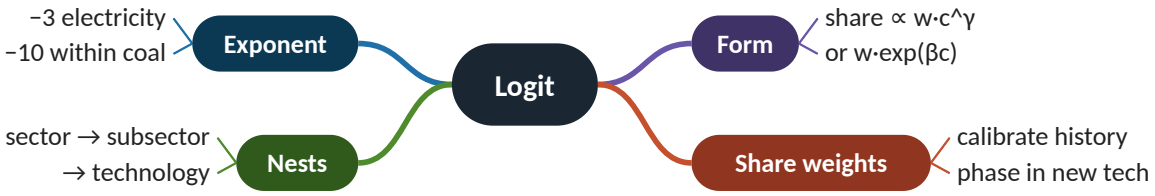
MODULE 3 OF 8

2.3 Choice: the logit

Why the cheapest option wins the most, but never everything.

HOOK

Coal at $60, gas at $70, solar at $80. A cost-minimising model builds 100% coal. GCAM, with its default electricity exponent, builds a mix. The reason is honest humility: real costs vary by site, by investor and by things no dataset records, so a logit spreads the market by how much cheaper each option is.



What this component represents

Every time GCAM must split a demand among competitors (fuels for a boiler, technologies for power, crops for land, modes for travel) it uses a **logit choice**. It turns costs into market shares smoothly: a small cost gap gives a small share gap, and no option jumps from 0% to 100% because of a cent.

How GCAM represents it

Two forms are documented (choice). The **relative-cost logit** (Clarke & Edmonds 1993) is the one used for most energy choices; the absolute-cost form exists too, and GCAM derives its β from the exponent the user enters.

$$s_i = \alpha_i \, p_i^{\gamma} \,/\, \Sigma_j \, \alpha_j \, p_j^{\gamma} \quad (\text{relative cost}, \gamma < 0) \tag{5}$$

$$s_i = \alpha_i \, e^{\beta \, p_i} \,/\, \Sigma_j \, \alpha_j \, e^{\beta \, p_j} \quad (\text{absolute cost}) \tag{6}$$

The **share weights** α are calibrated so the model reproduces base-year shares, which absorbs unmeasured preferences and costs. They also phase in new technologies: low when a technology first appears, rising to neutral. The documented reference values are γ = −3 for the electricity subsector choice and γ = −10 for technologies inside coal (choice). Choices are **nested**: sector → subsector → technology (→ cooling system for thermal power) (details_energy).

SIMPLE CALCULATION

Coal $60, gas $70, solar $80 per MWh, equal share weights, γ = −4. What share does each get?

1. Coal: $60^{-4} = 1 \div 12{,}960{,}000 = 7.716 \times 10^{-8}$.
2. Gas: $70^{-4} = 1 \div 24{,}010{,}000 = 4.165 \times 10^{-8}$.
3. Solar: $80^{-4} = 1 \div 40{,}960{,}000 = 2.441 \times 10^{-8}$.
4. Sum = 14.322×10^{-8}. Shares = 7.716/14.322, 4.165/14.322, 2.441/14.322.

Answer: coal 53.9%, gas 29.1%, solar 17.0%.

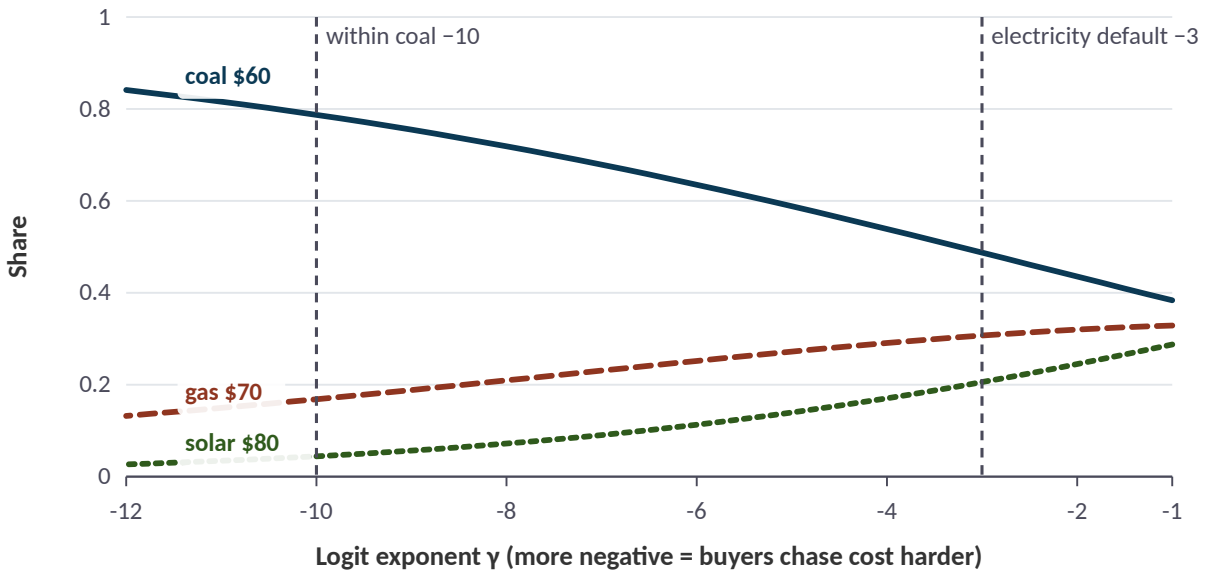


Figure 2.6 — the exponent is a dial for how hard buyers chase the cheapest. Shares for the same three costs as γ moves from −1 (almost equal shares) to −12 (coal dominates). Notice the two documented reference values: −3 keeps a broad mix, −10 nearly picks a winner.

Who it talks to

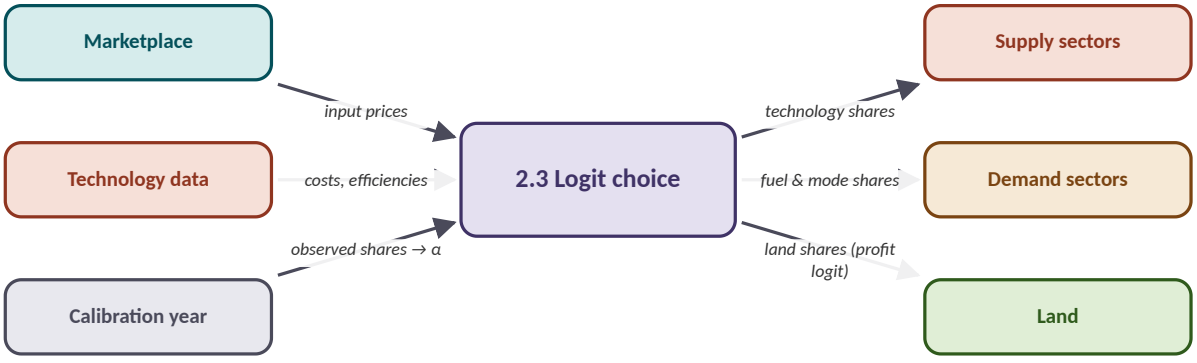


Figure 2.7 — interaction map. The logit is the shared machinery inside nearly every other box.

BUILD IT

Script: `python/m23_logit.py` (same logic in `R/m23_logit.R`). Key lines:

```
techs, costs = ["coal", "gas", "solar"], [60, 70, 80]
for g in (-1, -4, -10):
    print(f"relative logit, exponent {g:>3}: ", {k: round(float(v), 3) for k, v in
zip(techs, relative_logit(costs, g))})

# Share weights calibrate the base year: they make the logit reproduce observed shares.
observed = np.array([0.30, 0.50, 0.20])
g = -4
w = observed / (np.array(costs, float) ** g)
w = w / w[0]
print("calibrated share weights:", {k: round(float(v), 3) for k, v in zip(techs, w)},
      "-> reproduces", [round(float(v), 3) for v in relative_logit(costs, g, w)])

# Nested logit: electricity first chooses fossil vs renewable, then within each nest.
fossil = relative_logit([60, 70], -6)          # coal vs gas inside the fossil nest
```

It prints:

```
relative logit, exponent  -1:  {'coal': 0.384, 'gas': 0.329, 'solar': 0.288}
relative logit, exponent  -4:  {'coal': 0.539, 'gas': 0.291, 'solar': 0.17}
relative logit, exponent -10:  {'coal': 0.787, 'gas': 0.169, 'solar': 0.044}
calibrated share weights: {'coal': 1.0, 'gas': 3.088, 'solar': 2.107} -> reproduces [0.3,
0.5, 0.2]
nested: fossil nest 0.674 (coal 0.482, gas 0.191), solar 0.326
```

Change this and predict first:

1. Raise solar's share weight from 2.107 to 4 in the calibrated case. Predict: which technology loses most share?
2. Flip to the absolute-cost logit with b = −0.1. Is the coal share bigger or smaller than with γ = −4? Why does the absolute form care about dollar gaps and the relative form about ratios?

IN GCAM

Exponents and interpolation rules live in gcamdata assumption files such as `A23.subsector_logit.csv` and `A23.subsector_interp.csv` (the A21–A26 families) (inputs_supply). In the XML inputs a subsector carries `<share-weight>` by year and the sector a `<logit-exponent>`. Lab 8's CheapSolar add-on changes costs, not exponents, and the logit does the rest. Output to look at: *elec gen by subsector* (course query file).

TRAP

“A share weight is a subsidy.” It is not a price: it is a calibrated **preference multiplier**. Changing it moves shares without changing any cost, so it is a powerful and easily abused lever. Always report share-weight changes as assumptions.

PICTURE THIS · STOP 3 OF THE ROUTE

Chooser's desk. A clerk sits behind a desk with three jars labelled coal, gas and solar. Each customer drops a marble in a jar, but the jars have **funnels**: cheaper jars have wider funnels. A big dial on the desk marked **γ** widens the gap between funnels; a sticky note on each jar (the share weight) says “remember last year”. *Mapping: logit shares; exponent = dial; share weights = calibration notes.*

TRY IT NOW

(90 s) Two options cost \$50 and \$100. With γ = −2, what share does the cheap one get? (Hint: $50^{-2} \div (50^{-2} + 100^{-2})$.) Now with γ = −6?

QUESTIONS TO ASK OF ANY RUN

1. Which logit exponents and share weights differ from the release defaults?
2. Is a new technology held back by its phase-in share weights rather than its cost?
3. Would the result survive a ±50% change in the exponent?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.

MODULE 4 OF 8

2.4 Energy supply

Resources, power plants, refineries and hydrogen: where energy comes from and what it costs.

HOOK

Since GCAM v7.3, wind and solar no longer need a "backup" gas turbine bolted on. Instead each gets a **value factor** that shrinks as its market share grows: the 30th percent of solar is worth less to the grid than the 3rd (en_technologies; change proposal 394). The model now lets renewables limit themselves.



What this component represents

The supply side turns primary resources (oil, gas, coal, uranium, biomass, wind, sun, water) into the fuels and electricity that demand sectors buy. It decides **which technologies** do the converting and **what they cost**, including any carbon price on what they emit.

How GCAM represents it

Depletable resources use graded supply curves: cheap grades first, dearer ones as cumulative use grows; technical change can lower extraction cost (supply_energy). **Renewable resources** are annual flows on an upward curve (supply_energy):

$$Q = Q_{max} \cdot P^{b} \,/\, (\mathrm{MidPrice}^{b} + P^{b}) \tag{7}$$

Electricity is nested: fuel subsector → technology → cooling system. Each technology's cost is the sum of its parts (supply_energy, en_technologies):

$$C = \text{non-energy cost} + \Sigma\ \text{fuel price} \times \text{input coefficient} + \Sigma\ \text{carbon price} \times \text{emissions} - \text{secondary-output value} \tag{8}$$

$$\text{Effective cost of wind or solar} = \mathrm{LCOE} \div \mathrm{VF}, \quad \mathrm{VF} = (\text{intercept} + \text{slope} \times \text{share}) \div 0.903 \tag{9}$$

Documented value-factor coefficients: solar PV intercept 0.972, slope −1.443; onshore wind 0.91, −1.2; offshore wind 0.946, −1.661, each divided by 0.903 (the non-VRE average). They come from US ReEDS runs and may break down above about 75% VRE; storage is not modelled explicitly (change proposal 394). Plant-level costs, efficiencies and capacity factors are in gcamdata CSVs (NREL ATB 2019 data; updated in v8.8), not on the doc pages (inputs_supply). Other supply sectors: refining (oil, biomass liquids, coal- and gas-to-liquids), gas processing, hydrogen (10 central and 2 forecourt routes), district heat, and CCS as a removal fraction sent to a storage market.

SIMPLE CALCULATION

A gas plant: capital $1,000/kW, fixed O&M $25/kW-yr, capacity factor 55%, gas $6/GJ, efficiency 55%, 0.0561 tCO2/GJ, 7% discount rate over 30 years. Cost at $50/t CO2?

1. Capital recovery factor = $0.07 \times 1.07^{30} \div (1.07^{30} - 1) = 0.0806$.
2. MWh per kW per year = 0.55 × 8.76 = 4.818. Capital + O&M = (1,000 × 0.0806 + 25) ÷ 4.818 = 21.9 $/MWh.
3. Fuel = 6 $/GJ × 3.6 GJ/MWh ÷ 0.55 = 39.3 $/MWh.
4. Carbon = 0.0561 × 3.6 ÷ 0.55 × 50 = 18.36 $/MWh. Total = 21.91 + 39.27 + 18.36 = 79.55.

Answer: 79.5 $/MWh (61.2 without the carbon price). Illustrative inputs.

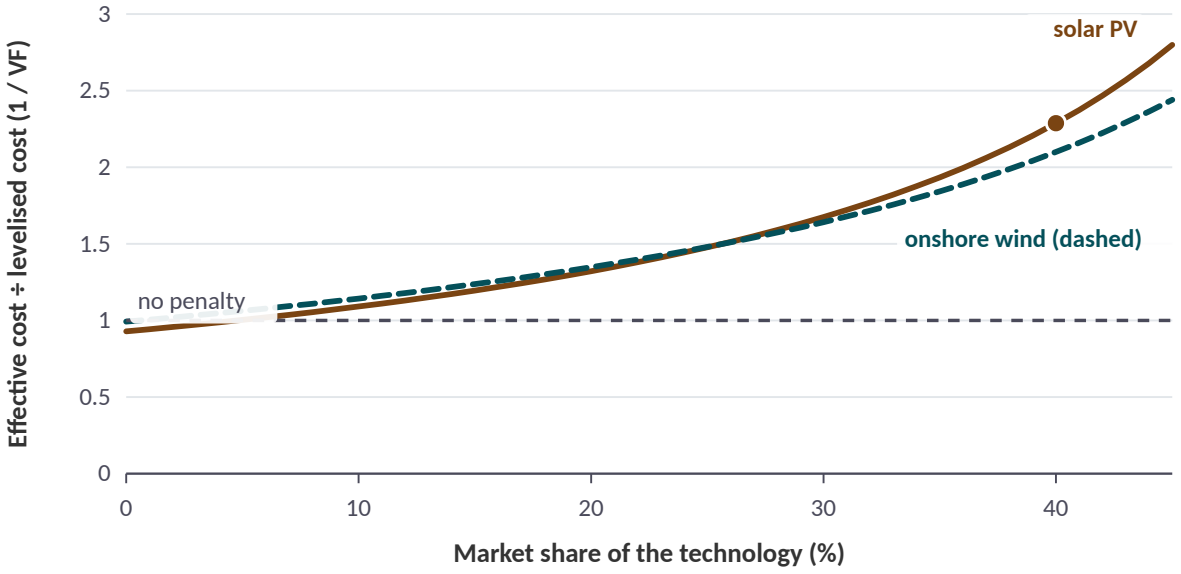


Figure 2.8 — the value-factor penalty (equation 9). 1/VF multiplies the levelised cost. Notice it starts slightly below 1 and climbs steeply: at 40% share solar PV's effective cost is about 2.3 times its levelised cost (onshore wind 2.1). Computed from the documented coefficients.

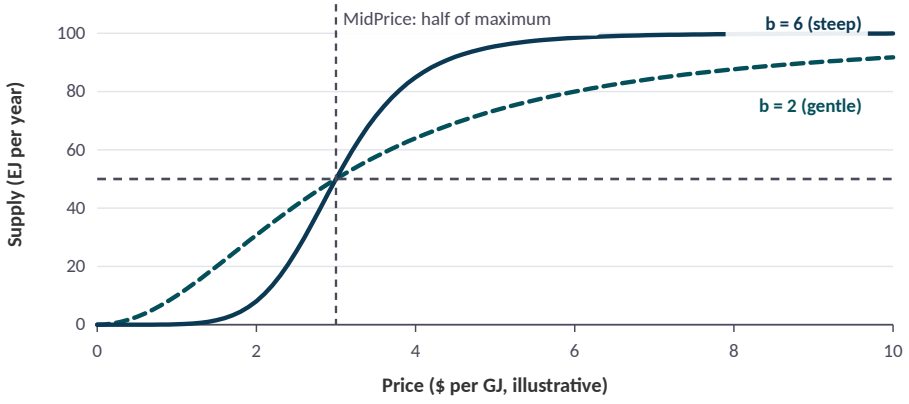


Figure 2.9 — renewable supply curve (equation 7) with Q_{max} = 100 EJ and MidPrice = 3 $/GJ. Notice *b* sets how sharply supply turns on around the mid-price. Illustrative parameters.

Who it talks to

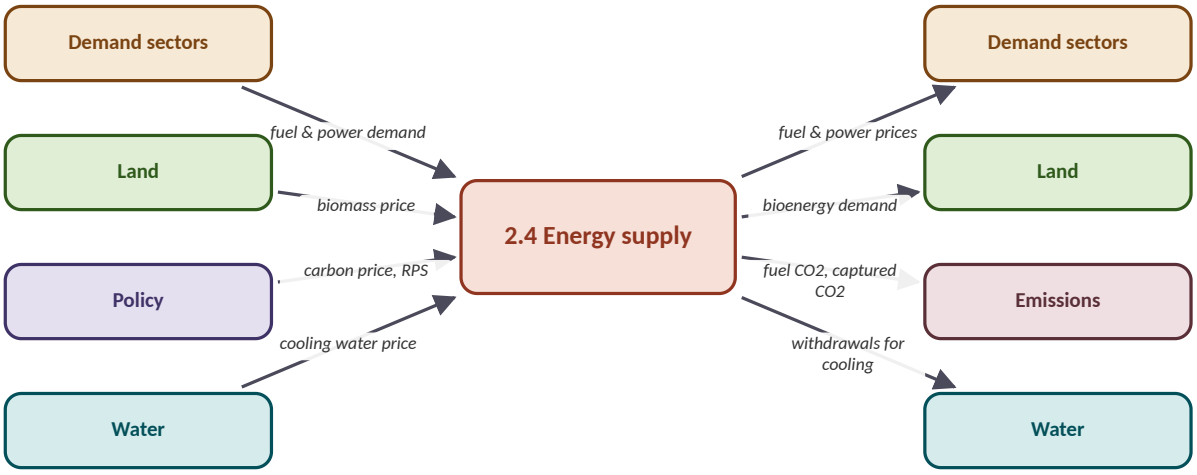


Figure 2.10 — interaction map. Supply is the busiest junction: four components send to it and it sends back to all four.

BUILD IT

Script: `python/m24_supply.py` (same logic in `R/m24_supply.R`). Key lines:

```
def lcoe(capex, fom, cf, fuel_price=0.0, eff=1.0, ef=0.0, carbon=0.0, r=0.07, n=30):
    """capex $/kW, fom $/kW-yr, cf capacity factor, fuel $/GJ, eff output/input, ef tCO2/GJ
fuel, carbon $/t."""
    mwh = cf * 8.76                                  # MWh produced per kW per year
    cap = (capex * crf(r, n) + fom) / mwh            # $/MWh
    fuel = fuel_price * 3.6 / eff                    # 3.6 GJ per MWh
    co2 = ef * 3.6 / eff * carbon
    return cap + fuel + co2
```

It prints:

```
carbon   0 $/t: gas  61.2 $/MWh, solar  45.4 $/MWh
carbon  50 $/t: gas  79.5 $/MWh, solar  45.4 $/MWh
carbon 100 $/t: gas  97.9 $/MWh, solar  45.4 $/MWh
resource price at 200, 800, 1400 EJ: [3.6, 6.0, 11.0] $/GJ
solar capex after 3 doublings at 20%: 460.8 $/kW
```

Change this and predict first:

1. Halve solar's capacity factor to 0.11 (a cloudy region). Predict the new solar cost before running.
2. At what carbon price does the gas plant reach 100 $/MWh? Solve by hand from the printed numbers, then check.

IN GCAM

Lab 8 adds a **CheapSolar** add-on XML after the reference inputs (a later file overrides an earlier one) and compares *elec gen by subsector* against Reference (user-guide). The new intermittency approach is selected by the XML tag `intermittent-technology`; the legacy one by `backup-intermittent-technology`; gcamdata switch `energy.ELEC_USE_BACKUP` (default FALSE). Prices of fuels are in 1975$/GJ (outputs_prices).

TRAP

“GCAM learns: costs fall because capacity grows.” In the released model, future technology costs are **inputs** (by year, by scenario). Learning curves are a way to *choose* those inputs, not something the model computes during a run.

PICTURE THIS · STOP 4 OF THE ROUTE

Power hall. Rows of turbines with price tags hanging from them. Each tag has three stickers: a brick (capital), a flame (fuel) and a black cloud (carbon). Solar panels on the roof carry a shrinking **magnifying glass**: the more panels there are, the smaller each one looks to the buyer. *Mapping: cost = capital + fuel + carbon; value factor falls with share.*

TRY IT NOW

(90 s) Using equation 9 for solar PV, compute VF at 0% and at 30% share. By what factor does the effective cost rise between the two?

QUESTIONS TO ASK OF ANY RUN

1. Which cost data year and scenario (core, advanced, low) are the technologies on?
2. Is intermittency handled by value factors (default) or the legacy backup method?
3. Are carbon storage and bioenergy limits binding?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.
4. **2.4 Supply:** cost = non-energy + fuel + carbon; resources get dearer as used; VRE value falls with share.

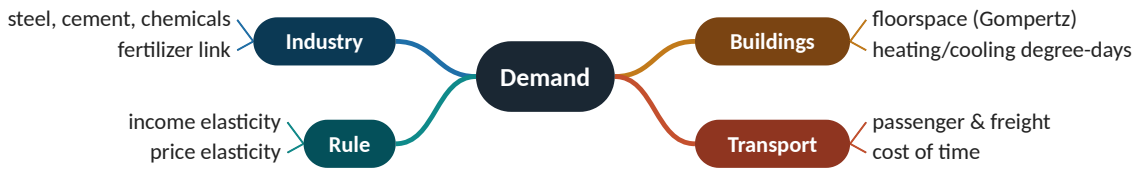
MODULE 5 OF 8

2.5 Energy demand

Buildings, industry and transport: people want warm rooms, steel and trips, not joules.

HOOK

In GCAM, a faster train can win passengers from cars even if it costs more per kilometre, because the model charges every traveller for their **time**, valued from the wage. As incomes rise, time gets dearer and fast modes win (demand_energy).



What this component represents

Demand sectors turn income and prices into **energy services** (floor space heated, tonnes of steel, passenger-kilometres), then choose fuels and technologies to provide them with logits (module 2.3).

How GCAM represents it

Buildings grow floor space per person with income on a saturating (Gompertz) curve, and heating and cooling services depend on degree-days, shell efficiency and income; residential demand is split into 10 income groups since v7.1 (demand_energy). **Industry** covers iron and steel, chemicals, aluminium, cement, fertilizer and other industry, plus construction, mining and agricultural energy use. Industry and transport service output grow with income and price (demand_energy):

$$D_t = D_{t-1} \cdot (Y_t/Y_{t-1})^{\alpha} \cdot (P_t/P_{t-1})^{\beta} \cdot (N_t/N_{t-1}) \tag{10}$$

Transport has four final demands: long-distance passenger air, other passenger, international shipping and other freight. Passenger nests have up to five levels (mode, sub-mode, size class, drivetrain, plus non-motorised modes) (demand_energy). Costs per passenger- or tonne-km:

$$P_j = (P_{fuel} \cdot I_j + N_j) \;/\; L_j \quad (I = \text{fuel intensity}, N = \text{non-fuel cost per vehicle-km}, L = \text{load factor}) \tag{11}$$

$$P_{mode} = \Sigma\, \alpha_j P_j + W \cdot V \;/\; S \quad (W = \text{wage rate from GDP per capita}, V = \text{time-value multiplier}, S = \text{speed}) \tag{12}$$

SIMPLE CALCULATION

Income per person doubles. Passenger-km have an income elasticity of 0.6 and a price elasticity of −0.3. What happens to demand, (a) if travel prices stay the same, (b) if they rise 50%?

1. Income factor = $2^{0.6}$ = 1.516.
2. (a) Demand = 100 × 1.516 = 151.6 (+52%).
3. Price factor = $1.5^{-0.3}$ = 0.885.
4. (b) Demand = 100 × 1.516 × 0.885 = 134.2 (+34%).

Answer: 151.6 and 134.2 (from a base of 100). Illustrative elasticities.

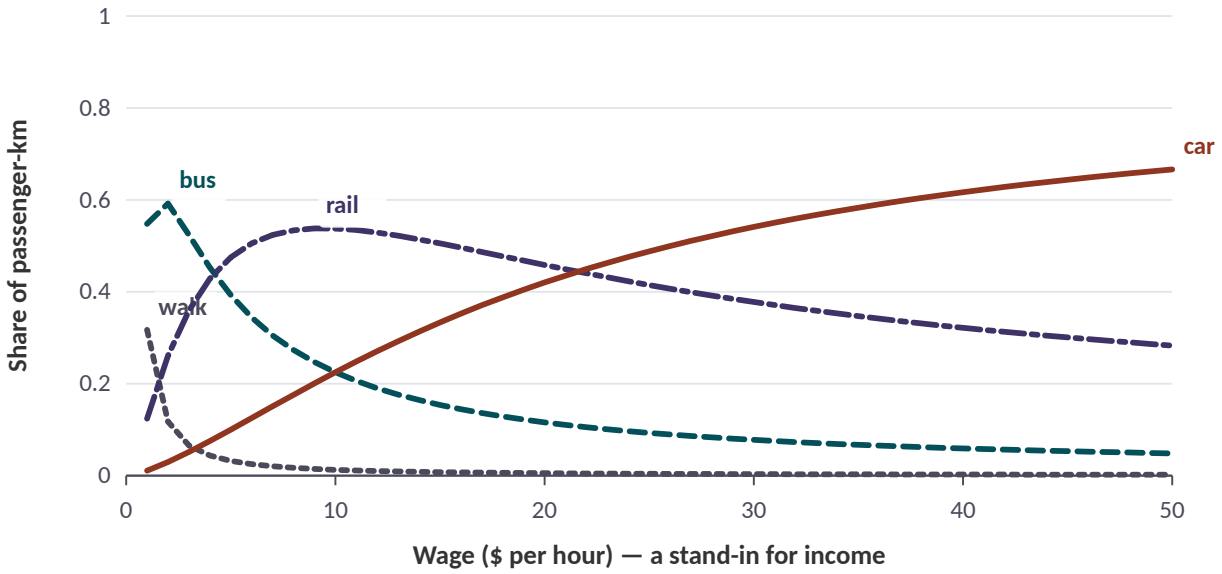


Figure 2.11 — the cost of time reshapes travel. Toy mode choice with equation 12 (value of time = half the wage; walk 5 km/h, bus 15, rail 30, car 60 door-to-door; logit γ = −3). Notice bus leads at low wages, rail in the middle, car at high wages, even though money costs never change. Illustrative numbers from `m25_demand.py`.

Who it talks to

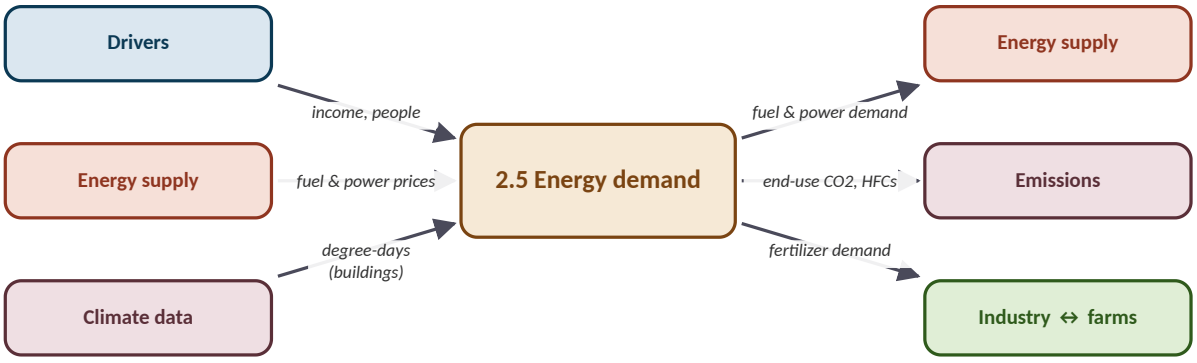


Figure 2.12 — interaction map. Demand pulls energy through the supply side and pushes emissions out.

BUILD IT

Script: `python/m25_demand.py` (same logic in `R/m25_demand.R`). Key lines:

```
modes = {"walk": (0.00, 5), "bus": (0.05, 15), "rail": (0.12, 30), "car": (0.30, 60)}   # $/pkm, door-to-door km/h
def mode_shares(wage, g=-3.0):
    vot = 0.5 * wage                              # value of time: half the wage, a common assumption
    gc = np.array([m + vot / s for m, s in modes.values()])
    x = gc ** g
    return {k: round(float(v), 3) for k, v in zip(modes, x / x.sum())}, [round(float(v), 3) for v in gc]

for wage in (2, 10, 40):                          # $ per hour
    sh, gc = mode_shares(wage)
```

It prints:

```
passenger-km if income doubles, price same: 151.6
...and if fuel makes travel 50% dearer:      134.2
wage  2 $/h -> generalised cost [0.2, 0.117, 0.153, 0.317] -> shares {'walk': 0.118, 'bus': 0.592, 'rail': 0.261, 'car': 0.03}
wage 10 $/h -> generalised cost [1.0, 0.383, 0.287, 0.383] -> shares {'walk': 0.013, 'bus': 0.225, 'rail': 0.538, 'car': 0.225}
wage 40 $/h -> generalised cost [4.0, 1.383, 0.787, 0.633] -> shares {'walk': 0.002, 'bus': 0.059, 'rail': 0.322, 'car': 0.617}
```

Change this and predict first:

1. Double the car's money cost to 0.60 $/pkm (a fuel tax). Predict the car share at wage 40 before running.
2. Set the value of time to the full wage instead of half. Which mode gains at wage 10?

IN GCAM

Look at *primary energy consumption by region (direct equivalent)* in the course queries, and in ModelInterface the standard queries for final energy by sector and transport service output by mode (outputs_quantity). Transport data enter from the UCD transportation database via SSP-specific files such as `UCD_trn_data_SSP*.csv` (ssp). Go 3 opens the transport sector fully.

TRAP

"A price elasticity of −0.3 means a 50% price rise cuts demand by 15%." Elasticities multiply as powers: $1.5^{-0.3} = 0.885$, an 11.5% cut. The linear rule only works for small changes.

PICTURE THIS · STOP 5 OF THE ROUTE

Demand floor. A busy hall where shoppers queue for warm rooms, steel beams and train tickets. Every shopper wears a **wristwatch with a price tag** on it; the richer the shopper, the bigger the tag, and they jump into the fastest vehicle on the conveyor. *Mapping: services, income elasticity, value of time.*

TRY IT NOW

(60 s) A car costs $0.30 per km and goes 60 km/h; rail costs $0.12 and goes 30 km/h. Value of time is $10 per hour. Which has the lower generalised cost per km?

QUESTIONS TO ASK OF ANY RUN

1. What income and price elasticities drive the service demand?
2. Are you comparing service output (pkm, m^2) or final energy (EJ)? They answer different questions.
3. Which value-of-time and speed assumptions sit behind a transport mode shift?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.
4. **2.4 Supply:** cost = non-energy + fuel + carbon; resources get dearer as used; VRE value falls with share.
5. **2.5 Demand:** service demand grows with income and falls with price; transport adds the cost of time.

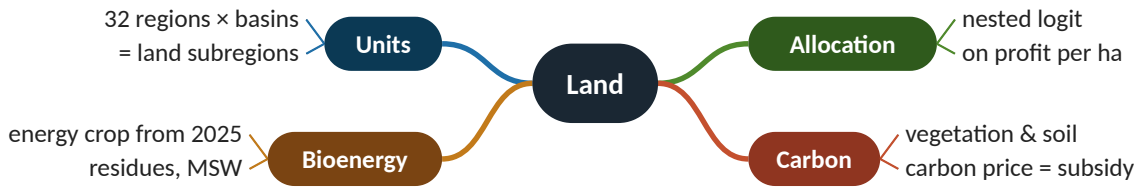
MODULE 6 OF 8

2.6 Land, food and bioenergy

Every hectare can grow food, fuel or forest; profit decides which.

HOOK

If the world wanted 100 EJ a year of bioenergy from energy grass at 10 tonnes per hectare, it would need about **556 million hectares**, roughly a third of today's cropland. Halve the yield and the land doubles. That is why IAMs that lean on bioenergy also move food prices.



What this component represents

The agriculture and land-use component (AgLU) decides how much land goes to each crop, pasture, forest and unmanaged land, how much each produces, and how much carbon is stored or released when land changes use.

How GCAM represents it

Land is resolved by **water-basin-based geographic land units** inside the 32 regions: the overview table lists 396 land subregions, while its text says “more than 300” (overview). Land sits in a **nest**: regional land → agro-forestry or non-agricultural → crops, pasture, forest… and every crop is split into irrigated/rainfed × high/low management, which allows price-induced intensification (land). Shares follow a logit on **profit** with a positive exponent (land):

$$s_i = (\lambda_i \, \pi_i)^{\rho} \; / \; \Sigma_j \, (\lambda_j \, \pi_j)^{\rho}, \quad a_i = a_{parent} \cdot s_i \tag{13}$$

$$\pi = (\text{price} + \text{subsidy} - \text{variable cost} - \text{input costs}) \times \text{yield} \quad \text{yield}_t = \text{yield}_{t-1} \, (1 + \text{APG})^{\Delta t} \tag{14}$$

λ is a calibrated profit scaler that makes the base year match observations (land, supply_land). Numerical logit exponents are not on the doc pages. **Land carbon**: when land changes, emissions are $E_t = A_t D_t - A_{t-1} D_{t-1}$; releases are immediate, but regrowth follows a sigmoid with a mature age (about 30–100 years for forests). With a carbon price on land, owners are **paid for holding carbon**: carbon price × carbon density, discounted and annualised (land).

SIMPLE CALCULATION

Contested land can grow wheat ($200/t, cost $120/t, 3 t/ha) or energy grass ($60/t, cost $30/t, 10 t/ha). Share the land by profit with exponent 1.

1. Wheat profit = (200 − 120) × 3 = $240 per ha.
2. Grass profit = (60 − 30) × 10 = $300 per ha.
3. Total = 240 + 300 = 540.
4. Shares: wheat 240 ÷ 540 = 0.444; grass 300 ÷ 540 = 0.556.

Answer: wheat 44%, grass 56% of the contested land. Illustrative.

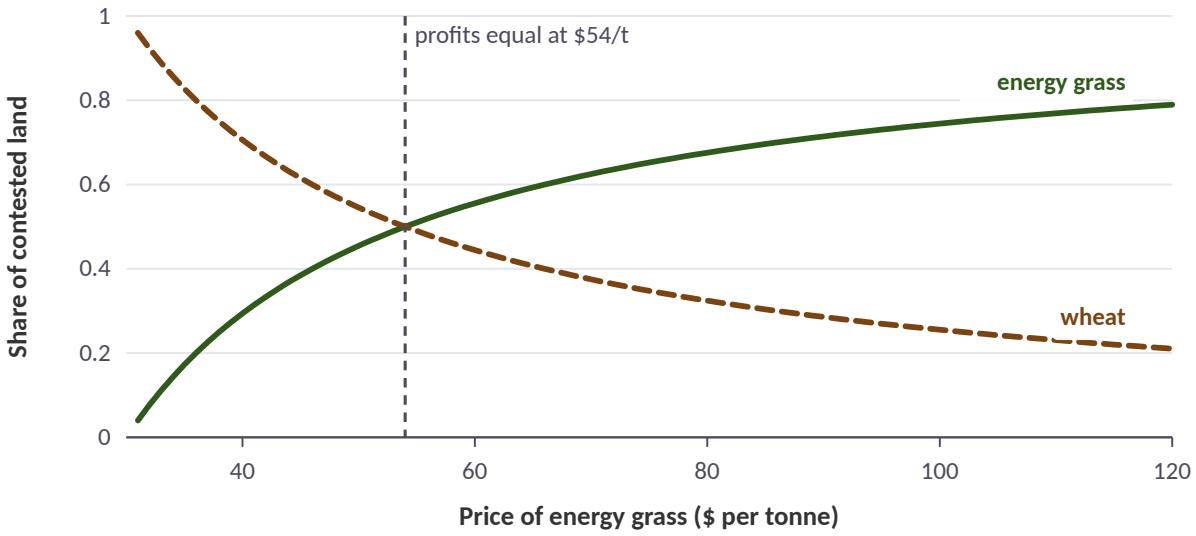


Figure 2.13 — land follows profit. As the energy-grass price rises, its share of contested land rises smoothly (equation 13, ρ = 1). Notice the crossing at $54/t, where (p − 30) × 10 = 240.

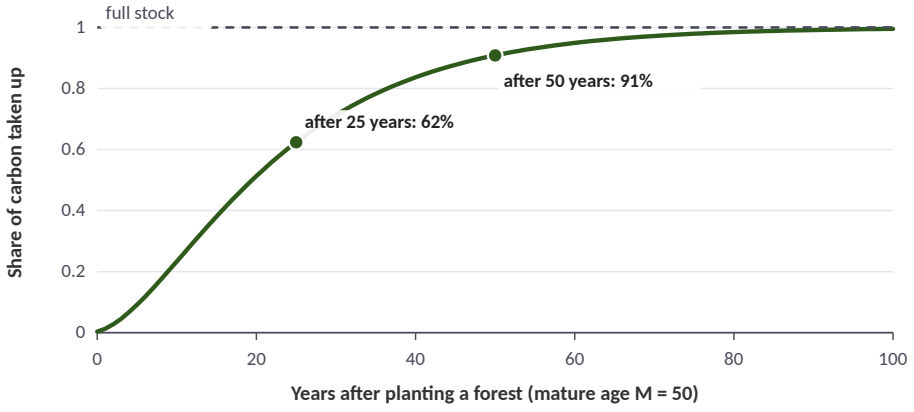


Figure 2.14 — forests take decades to soak up carbon. The documented regrowth sigmoid with mature age M = 50 years: 62% of the carbon is taken up after 25 years and 91% after 50. Notice a carbon price pays for a stock that arrives slowly, while clearing releases it at once.

Who it talks to

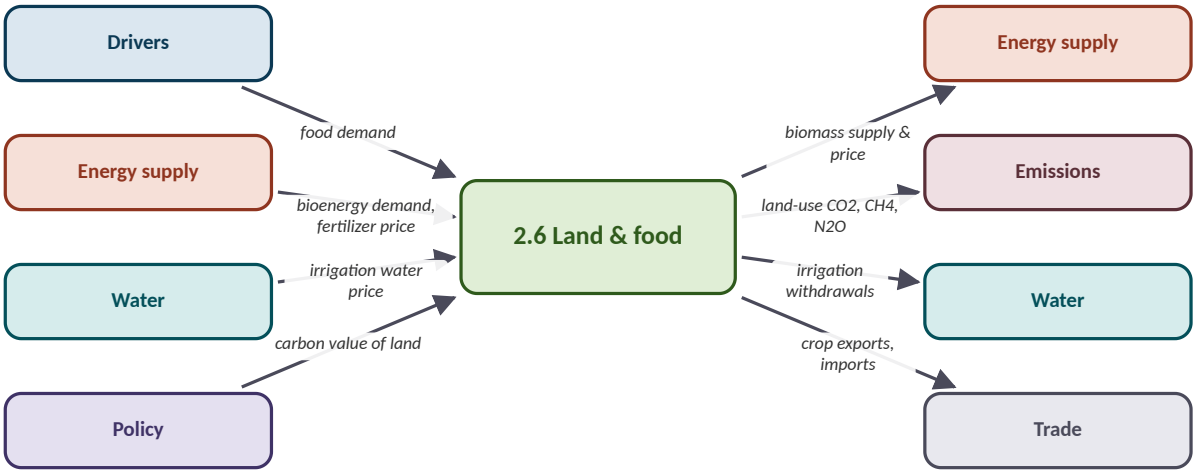


Figure 2.15 — interaction map. Land is where energy policy meets food prices.

BUILD IT

Script: `python/m26_land.py` (same logic in `R/m26_land.R`). Key lines:

```
for grass_price in (40, 60, 90):
    pr = [profit(200, 120, 3), profit(grass_price, 30, 10)]
    s = land_shares(pr)
    print(f"grass ${grass_price}/t: profits wheat {pr[0]:.0f}, grass {pr[1]:.0f} $/ha ->
shares wheat {s[0]:.3f}, grass {s[1]:.3f}")
```

It prints:

```
grass $40/t: profits wheat 240, grass 100 $/ha -> shares wheat 0.706, grass 0.294
grass $60/t: profits wheat 240, grass 300 $/ha -> shares wheat 0.444, grass 0.556
grass $90/t: profits wheat 240, grass 600 $/ha -> shares wheat 0.286, grass 0.714
 10 EJ at 10 t/ha ->   55.6 million ha
100 EJ at 10 t/ha ->  555.6 million ha
100 EJ at  5 t/ha -> 1111.1 million ha
```

Change this and predict first:

1. Change the exponent from 1 to 3 (land use more responsive). Predict the grass share at $60/t: closer to 50% or further away?
2. Double wheat yield to 6 t/ha (intensification). How much land does wheat win back at $60 grass?

IN GCAM

The course query *aggregated land allocation* gives land by type; lab 9 maps it with `rmap` using the built-in `mapGCAMBasins`/`mapGCAMLand` shapes. Land carbon state is set in gcamdata by `aglu.CARBON_STATE` (default “q3_value”) (land). Protected land: the land page describes a suitability × IUCN approach, while the policies page mentions 90% of non-commercial ecosystems; treat the land page as the current method.

TRAP

“Bioenergy is carbon-neutral, so a carbon price only helps.” In GCAM the land that grows it was doing something else: clearing forest releases carbon immediately, and food prices rise. The carbon value of land (equation 14's subsidy) is what stops a carbon price from cutting down forests for fuel.

PICTURE THIS · STOP 6 OF THE ROUTE

Farm yard. A muddy yard where wheat sheaves and tall grass bales **arm-wrestle** over a fence; the referee holds a **profit calculator**. Behind them an oak tree slowly fills a piggy bank with coins labelled “carbon”, one coin a year. *Mapping: land shares by profit; carbon subsidy accrues slowly.*

TRY IT NOW

(60 s) How much land does 30 EJ of energy grass need at 12 t/ha and 18 GJ per tonne? (Hint: 30×10^9 GJ ÷ (12 × 18).)

QUESTIONS TO ASK OF ANY RUN

1. Is land carbon priced, and at what multiple of the fossil carbon price?
2. Which protected-land assumption is on?
3. Is bioenergy limited by the default negative-emissions budget or externality curve?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.
4. **2.4 Supply:** cost = non-energy + fuel + carbon; resources get dearer as used; VRE value falls with share.
5. **2.5 Demand:** service demand grows with income and falls with price; transport adds the cost of time.
6. **2.6 Land:** land is shared by profit per hectare in nested logits; carbon prices pay for land carbon.

MODULE 7 OF 8

2.7 Water

235 basins, three sources and a price that tells farms and power plants how scarce water is.

HOOK

In GCAM, irrigation water is charged only **5%** of the basin's general water price, a subsidy multiplier based on OECD data (supply_water). The model therefore reproduces a familiar real-world pattern: farms use most of the water, and they feel the least of its price.



What this component represents

Water is both a demand (for crops, power plant cooling, industry, mining, livestock and towns) and a constrained supply that differs by river basin. When a basin runs short, its water price rises and every user in it adjusts.

How GCAM represents it

GCAM uses **235 basins** (common_assumptions; the overview text says 233). Supply comes in tiers: **renewable water** from Xanthos runoff, cheap up to the accessible amount; **non-renewable groundwater** as a graded depletable resource (Superwell costs, 20 grades per region-basin); and **desalination**, in core runs only for municipal and industrial use (supply_water). A basin shadow price (1975$/m³) clears the market. Demand covers six sectors; power plants choose among five cooling systems with a cost-based logit (demand_water).

$$\text{Accessible water QA} = \max(0, \min(\text{QT} - \text{EFR}, \text{QB} - \text{EFR} + \text{RS})) \tag{15}$$

$$\text{pcW}_t = \text{pcW}_{t-1} \cdot (\text{pcGDP ratio})^{0.37} \cdot (\text{price ratio})^{-0.33} \cdot (1 - \text{Tech}_t)\ (\text{municipal water}) \tag{16}$$

QT is streamflow, QB baseflow, EFR the environmental flow requirement and RS reservoir storage. Energy also needs water and water needs energy: reverse-osmosis desalination uses 2.75 kWh per m³, thermal desalination 58.3, groundwater abstraction 0.185 (demand_energy).

SIMPLE CALCULATION

A city's water use per person grows with income (elasticity 0.37) and falls with price (−0.33). Income doubles and the water price rises 50%; ignore technology. Factor change?

1. Income factor = $2^{0.37}$ = 1.292.
2. Price factor = $1.5^{-0.33}$ = 0.875.
3. Combined = 1.292 × 0.875 = 1.131.

Answer: about 13% more water per person.

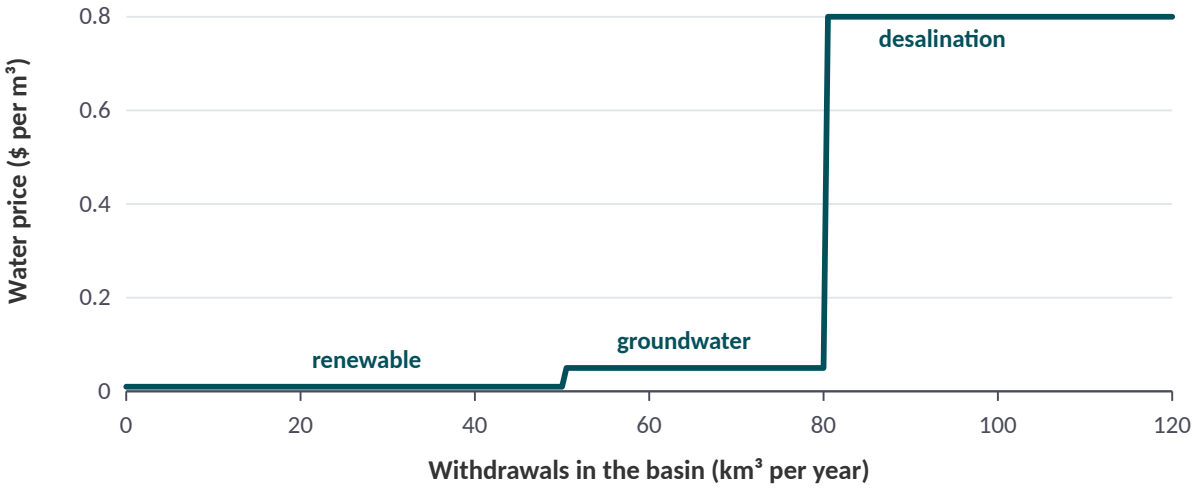


Figure 2.16 — the basin supply ladder. Toy tiers from `m27_water.py`: renewable water to 50 km³ at \$0.01/m³, groundwater to 80 km³ at \$0.05, then desalination at \$0.80. Notice the jumps: GCAM's real curves rise steeply beyond the accessible amount rather than in clean steps.

Who it talks to

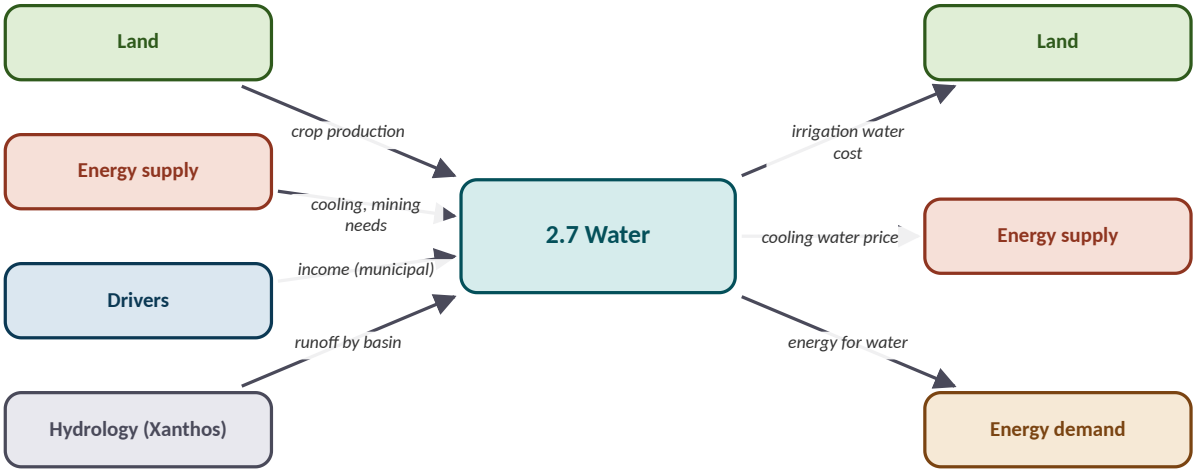


Figure 2.17 — interaction map. Water is a local market: the same price signal reaches farms and power plants in one basin.

BUILD IT

Script: `python/m27_water.py` (same logic in `R/m27_water.R`). Key lines:

```
def withdrawals(irrigation, municipal, power_twh, litres_per_kwh=2.0):
    power = power_twh * 1e9 * litres_per_kwh / 1e12      # km3 (1 km3 = 1e12 litres)
    return irrigation + municipal + power

for twh, lpk in ((100, 2.0), (300, 2.0), (300, 60.0)):   # dry cooling vs once-through
cooling
    w = withdrawals(40, 8, twh, lpk)
    print(f"{twh} TWh at {lpk:>4} L/kWh: withdrawals {w:6.1f} km3 -> water price
${water_price(w)}/m3")
```

It prints:

```
100 TWh at  2.0 L/kWh: withdrawals   48.2 km3 -> water price $0.01/m3
300 TWh at  2.0 L/kWh: withdrawals   48.6 km3 -> water price $0.01/m3
300 TWh at 60.0 L/kWh: withdrawals   66.0 km3 -> water price $0.05/m3
```

Change this and predict first:

1. Switch 300 TWh of power from once-through (60 L/kWh) to recirculating cooling at 4 L/kWh. Predict the water price.
2. Raise irrigation from 40 to 75 km^3 with dry cooling. Which tier sets the price?

IN GCAM

Course queries *water withdrawals by region* and *water withdrawals by basin*; lab 9 maps basins with `rmap::mapGCAMBasins` (demand_water). The irrigation subsidy multiplier is the gcamdata constant `water.IRR_PRICE_SUBSIDY_MULT` (supply_water).

TRAP

"Withdrawal" and "consumption" are not the same. A once-through cooled plant withdraws a lot but returns most of it; consumption is what evaporates or is embedded. GCAM reports both; compare like with like.

PICTURE THIS · STOP 7 OF THE ROUTE

Water tower. A tall tower with three taps stacked up its side: a wide **blue tap** at the bottom (river), a narrow **brown tap** (well), and a gold tap at the top (desalination) that dispenses one drop for a coin. A farmer gets a 95%-off voucher. *Mapping: tiers, rising price, irrigation subsidy.*

TRY IT NOW

(60 s) Desalinating 1 km^3 (10^9 m^3) by reverse osmosis at 2.75 kWh/m^3 needs how many TWh of electricity?

QUESTIONS TO ASK OF ANY RUN

1. Is the basin water market binding anywhere, or is water effectively free?
2. Are you reporting withdrawals or consumption?
3. Which basins move to groundwater or desalination, and when?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.
4. **2.4 Supply:** cost = non-energy + fuel + carbon; resources get dearer as used; VRE value falls with share.
5. **2.5 Demand:** service demand grows with income and falls with price; transport adds the cost of time.
6. **2.6 Land:** land is shared by profit per hectare in nested logits; carbon prices pay for land carbon.
7. **2.7 Water:** 235 basins; cheap renewable water first, then groundwater, then desalination.

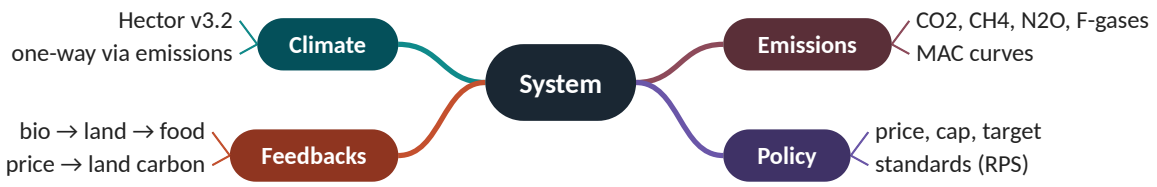
MODULE 8 OF 8

2.8 Emissions, climate and policy: the system together

Emissions leave every box, Hector turns them into warming, and a policy pulls on all the boxes at once.

HOOK

A carbon price and an emissions cap are two doors into the same room. Give GCAM a cap and it solves for the carbon price that meets it; give it that price and you get the same emissions back (policies). In the course toy model, a 50% cut in 2050 needs a price that starts tiny and grows 5% a year, because cheap options exist; the same model under a stiff fixed price leans heavily on bioenergy with CCS and land.



What this component represents

Emissions are the exhaust of every other component. The climate model converts them into concentrations, forcing and temperature. Policies are rules that change prices or limits so that emissions, and therefore temperature, change.

How GCAM represents it

Emissions: CO2 from fuel use × carbon coefficients, cement, fugitive and land-use change; non-CO2 gases from activity × emission factor, reduced by **marginal abatement cost (MAC)** curves that read the carbon price (emissions):

$$E_t = A_t \cdot F_{t0} \cdot (1 - \mathrm{MAC}(\mathrm{price}_t)) \tag{17}$$

Climate: each period GCAM passes emissions to **Hector v3.2**, which computes concentrations, radiative forcing and global temperature. The coupling is one-way: climate does not automatically change yields or runoff in core GCAM (hector). **Policy** types: a fixed carbon price path (`fixedTax`), an emissions constraint (GCAM solves for the price), a climate target (forcing, concentration or temperature via target-finder; slower), linked GHG markets with 100-year GWPs, renewable and biofuel standards, and land policies (policies, policies_examples). Carbon prices are reported in 1990$ per tonne of carbon (outputs_prices).

SIMPLE CALCULATION

A region makes 1,000 TWh from coal at 0.9 t CO2 per MWh. A landfill emits 2 Mt CH4; the carbon price makes 30% capture worthwhile on its MAC curve. Emissions?

1. Coal CO2 = 1,000 TWh × 0.9 Mt/TWh = 900 Mt CO2.
2. Methane after abatement = 2 × (1 − 0.30) = 1.4 Mt CH4 (equation 17).
3. Carbon price units: $100 per t CO2 = 100 × 44/12 = $367 per t C.

Answer: 900 Mt CO2 and 1.4 Mt CH4; and remember GCAM's carbon price is per tonne of *carbon*.

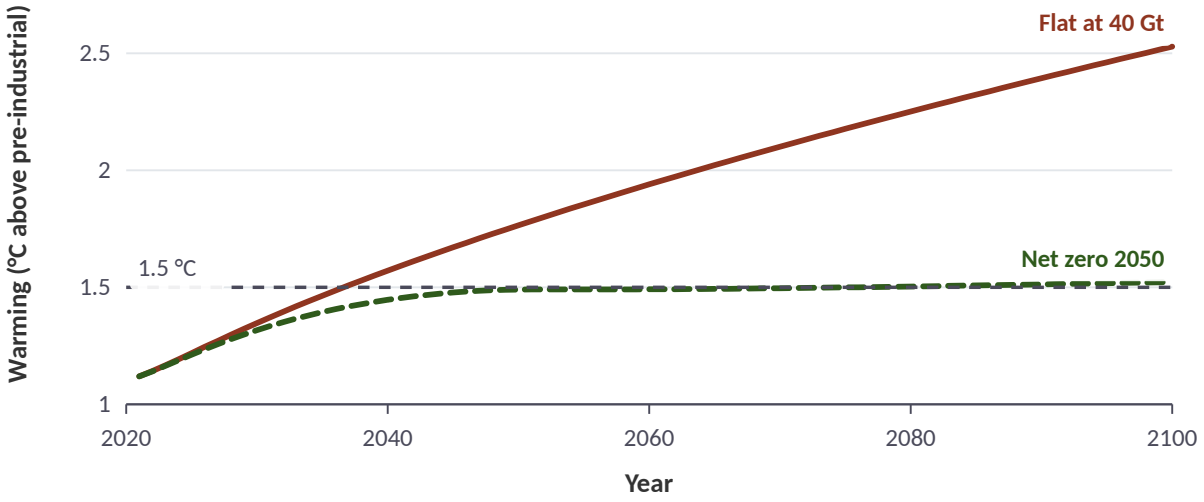


Figure 2.18 — emissions to warming with the course's toy climate model. Impulse-response carbon cycle plus a two-layer energy balance (`m28a_climate.py`; textbook parameters, not Hector). Flat 40 Gt: 2.53 °C in 2100 at 637 ppm; net zero in 2050: 1.52 °C. CO2 only.

Toy run (m28_system.py)	2050 CO2 (Gt)	solar share	bio land (Mha)	warming added by 2100
Reference	15.92	0.54	40.8	0.37 °C
Carbon price	-6.66	0.80	510.6	-1.04 °C
Cap -50% by 2050	9.76	0.67	67.0	0.19 °C

A one-region power-and-land toy. The carbon price starts at $25/t and grows 5% a year; the cap needs a far lower starting price because solar is already cheapening. Notice the fixed price overshoots into negative emissions with half a billion hectares of energy crops: precisely the land-for-bioenergy feedback of module 2.6.

The whole system

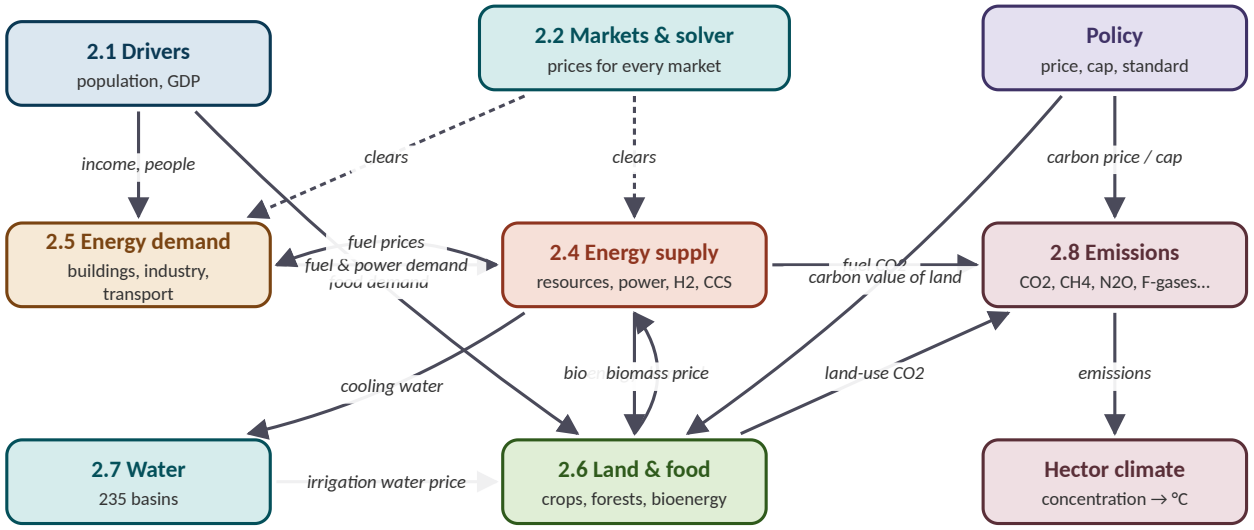


Figure 2.19 — every box and its main arrows. Dashed arrows: the marketplace clears every market. Follow any loop below on this figure.

Five feedback loops to trace in a real run

Loop	Mechanism (GCAM docs)	What to query
1. Bioenergy → land → food	Energy demand for biomass raises energy-crop profit; the land logit shifts cropland; other crop prices rise (land, supply_land)	aggregated land allocation; prices of all markets
2. Carbon price → land carbon → forests	Landowners are paid carbon price × carbon density; forest profit rises; land shifts to forest; crops intensify (land)	land allocation; LUC emissions
3. Electricity price → electrification	Transport cost = (fuel price × intensity + non-fuel)/load factor; cheaper power shifts drivetrain shares; more power demand raises its price (demand_energy)	final energy by fuel; elec gen by subsector
4. Water scarcity → cooling and irrigation	Basin price rises beyond accessible water; plants shift cooling; irrigated vs rainfed shares move (supply_water, demand_water)	water withdrawals by basin
5. VRE share → value factor	Higher solar or wind share lowers VF, raising effective cost, damping further growth (change proposal 394)	elec gen by subsector
6. Carbon price → non-CO2	MAC curves read the CO2 price; linked markets use 100-year GWPs (emissions, policies)	CO2 prices; non-CO2 emissions

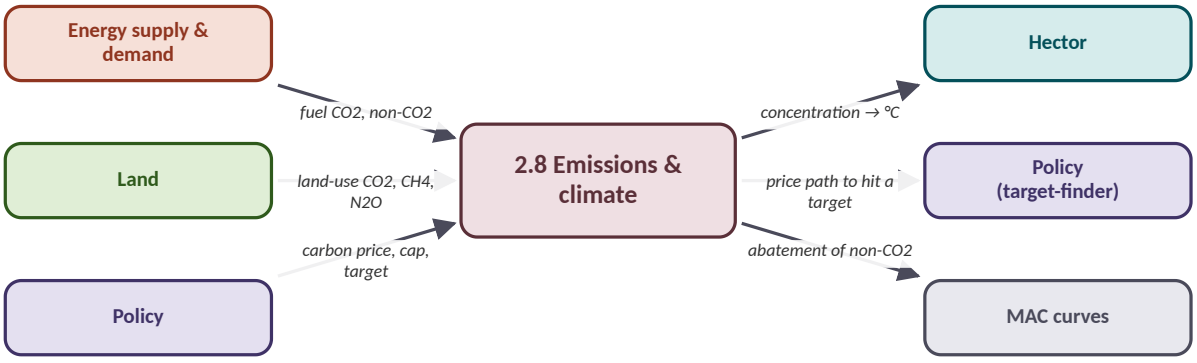


Figure 2.20 — interaction map. Emissions collect from everywhere; only the target-finder sends information back to prices.

BUILD IT

Script: `python/m28_system.py` (same logic in `R/m28_system.R`). Key lines:

```
ref = run(np.zeros(len(years)))
tax = run(25 * 1.05 ** (years - 2025))
i50 = list(years).index(2050)
target = 0.5 * ref[0]["co2"]
p0 = brentq(lambda p: run(p * 1.05 ** (years - 2025))[i50]["co2"] - target, 0.01, 5000)
cap = run(p0 * 1.05 ** (years - 2025))
```

It prints:

```
Reference         2050: CO2  15.92 Gt, solar share 0.54, bio land   40.8 Mha | warming added
by 2100 0.37 C
Carbon price      2050: CO2  -6.66 Gt, solar share 0.80, bio land  510.6 Mha | warming added
by 2100 -1.04 C
Cap -50% by 2050  2050: CO2   9.76 Gt, solar share 0.67, bio land   67.0 Mha | warming added
by 2100 0.19 C
The cap is met by a carbon price starting at 2.5 $/t in 2025 and growing 5%/yr: a price and
a cap are two doors to the same room.
```

Change this and predict first:

1. Lower LAND_COST from 0.08 to 0.02 (cheap land). Predict the bio land and 2050 CO2 under the fixed carbon price, then run.
2. Change the cap to −80% by 2050. Does the starting price rise a little or a lot? Explain with the logit (module 2.3).

IN GCAM

Lab 10 runs a carbon price (an add-on with `<ghgpolicy name="CO2"><market>…</market><fixedTax year="2025" fillout="1">…</fixedTax>`, pattern from policies_examples) and a net-zero-style constraint, then compares *CO2 emissions by region*, *CO2 prices* and *global mean temperature*. Lab 2 ran Hector on its own; lab 11 contrasts GCAM with an optimisation model (a linear program); lab 12 places your runs among other models' AR6 results.

RESEARCH NOTE

Contrast: optimisation models. MESSAGEix and TIMES choose the whole future at once to minimise total discounted cost, with perfect foresight; GCAM clears markets one period at a time. In an optimisation model a carbon cap's price is a **shadow price** (lab 11: $22.2/t for the 40 Mt cap); in GCAM it is a solved market price. The MESSAGEix Westeros tutorials (message-ix 3.11.1; needs GAMS) let you see the difference.

TRAP

“Hector feeds warming back into crop yields in GCAM.” Not in core GCAM: the coupling is one-way via emissions. Climate impacts must be imposed as inputs (e.g. Xanthos runoff, degree-days from `helios`) (hector).

PICTURE THIS · STOP 8 OF THE ROUTE

Chimney & control room. Every pipe in the factory joins one tall chimney. At the top sits a **thermometer the size of a lighthouse** (Hector). In the control room below, the manager has two levers welded together: one labelled “price”, one labelled “cap”. Pull either and both move. *Mapping: emissions collect, Hector converts, price and cap are duals.*

TRY IT NOW

(90 s) Convert $50 per tonne of CO2 into GCAM's units ($ per tonne of carbon). Then: under a cap, if a cheaper clean technology appears, does the solved carbon price rise or fall?

QUESTIONS TO ASK OF ANY RUN

1. Which gases are priced: CO2 only, or linked non-CO2 with GWPs?
2. Is the price per tonne of carbon or CO2, and in which dollar year?
3. Is warming from Hector's default parameters, and is that reported as such?

RECAP SO FAR

1. **2.1 Drivers:** population × income sets the scale of every demand; SSPs fix both.
2. **2.2 Markets:** GCAM finds prices that clear every market each period, with Broyden's method in log prices.
3. **2.3 Logit:** share = $w \cdot c^{\gamma} / \Sigma\, w \cdot c^{\gamma}$; γ sets how hard buyers chase the cheapest; share weights calibrate history.
4. **2.4 Supply:** cost = non-energy + fuel + carbon; resources get dearer as used; VRE value falls with share.
5. **2.5 Demand:** service demand grows with income and falls with price; transport adds the cost of time.
6. **2.6 Land:** land is shared by profit per hectare in nested logits; carbon prices pay for land carbon.
7. **2.7 Water:** 235 basins; cheap renewable water first, then groundwater, then desalination.
8. **2.8 System:** emissions go to Hector; a price or a cap moves every market at once.

Reading a GCAM run end to end

Go 2 opened each box separately. A real study reads them together. This order works for any pair of runs (a Reference and a policy scenario); each step names the module that explains it and the course query that shows it.

Step	Look at	Module	Query / file	Ask
1	Did both runs solve every period?	2.2	exe/logs/main_log.txt	Any unsolved period? Stop and fix first.
2	Same drivers in both runs?	2.1	GDP and population by region	If GDP differs, is the macro module on?
3	Policy signal	2.8	CO2 prices	Units: 1990$ per t C. Path shape?
4	Emissions	2.8	CO2 emissions by region	How big is the cut, and where?
5	Electricity mix	2.3, 2.4	elec gen by subsector	Which subsectors gain? Any VRE saturation?
6	Primary energy	2.4, 2.5	primary energy consumption by region	Did demand fall, or only the mix change?
7	Land	2.6	aggregated land allocation	Bioenergy and forest area; is food land squeezed?
8	Water	2.7	water withdrawals by basin	Which basins move? Cooling or irrigation?
9	Climate	2.8	global mean temperature	Difference from Reference by 2100.
10	Explain one loop	2.8 loops	two of the above	Write the causal chain in three sentences.

COMPARE TWO RUNS

A minimal comparison in R once both scenarios are in one project file (the pattern of labs 8 and 10):

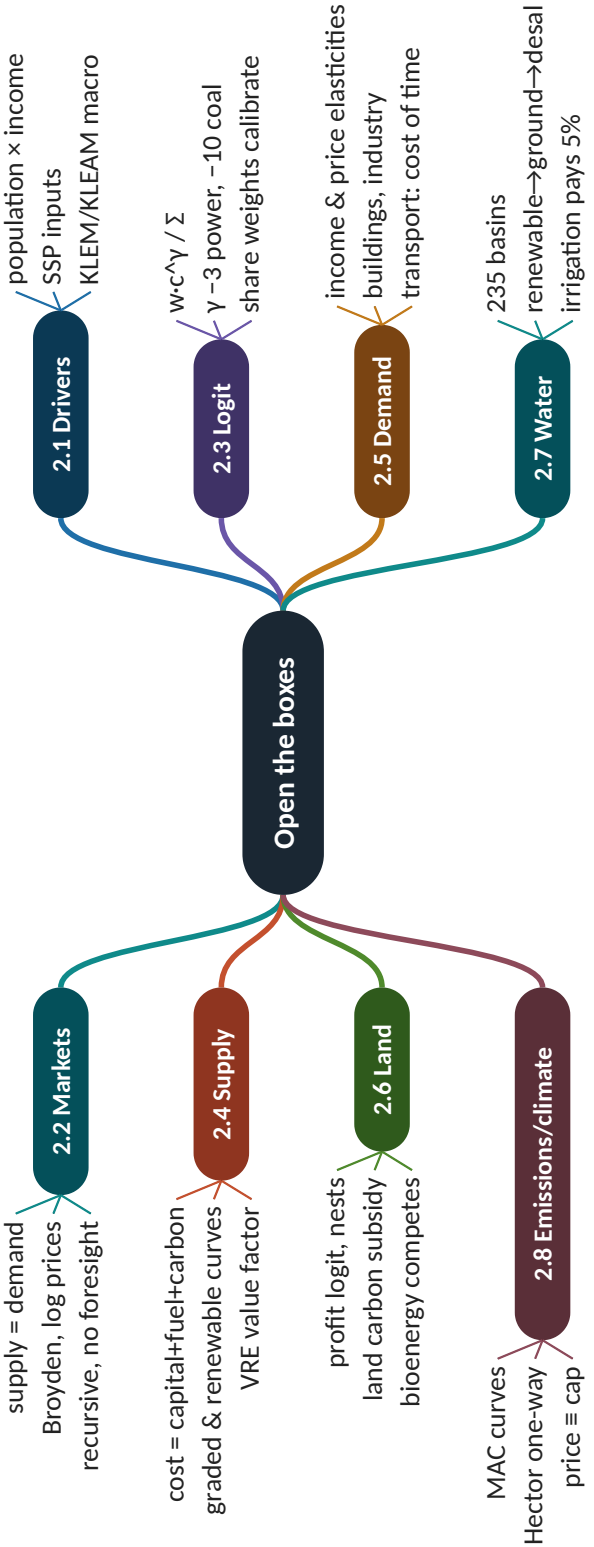
```
co2 <- getQuery(prj, "CO2 emissions by region")
diff <- co2 |>
  dplyr::group_by(scenario, year) |>
  dplyr::summarise(value = sum(value), .groups = "drop") |>
  tidyr::pivot_wider(names_from = scenario, values_from = value) |>
  dplyr::mutate(change = CarbonPrice - Reference)   # MtC per year
```

WHAT GO 2 DOES NOT YET DO

Go 2 explains mechanisms and reads results. It does not yet change the data system (gcamdata), run ensembles, or test robustness. Go 3 takes the transport sector as a full research case: its structure, data, calibration, scenario design, uncertainty analysis and write-up.

Master mind map

Turn the page sideways: every box in this handbook with the rules to remember.



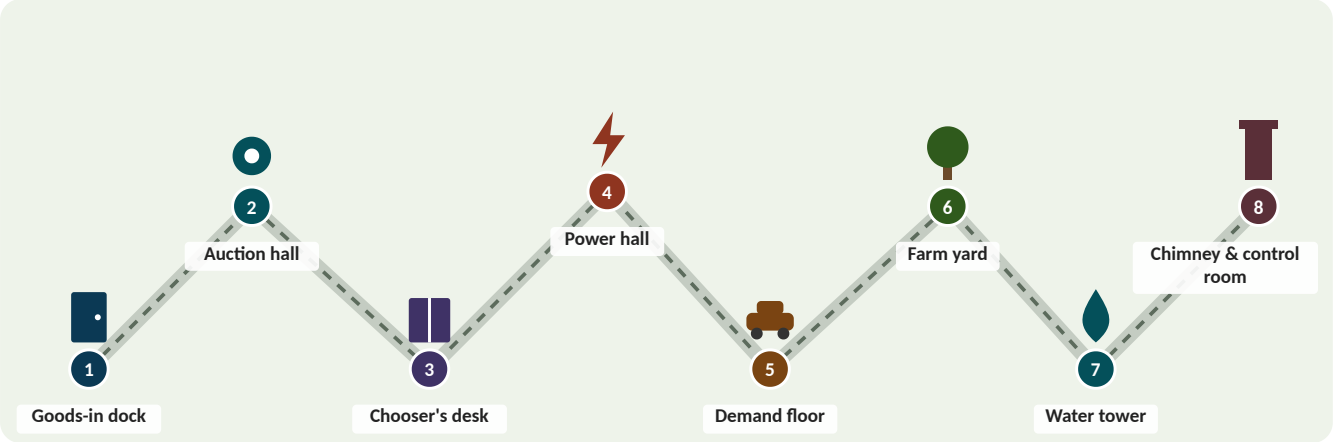
Connections and contrasts

Pair students confuse	The one fact that separates them
Relative vs absolute logit	Relative compares cost *ratios* (p^{γ}); absolute compares cost *differences* ($e^{\beta p}$). GCAM energy mostly uses relative.
Share weight vs cost	A share weight multiplies preference; a cost enters the price. Only costs respond to carbon prices.
Carbon price vs cap	Price fixed → emissions solved. Cap fixed → price solved. Same room, two doors.
GCAM vs optimisation (MESSAGEix)	Recursive market clearing vs whole-horizon cost minimisation with foresight.
Withdrawal vs consumption	Withdrawal = taken; consumption = not returned.
Land profit logit vs energy cost logit	Land uses profit with a positive exponent; energy uses cost with a negative one.
$ per t CO2 vs $ per t C	× 44/12 = 3.667. GCAM reports carbon prices per tonne of C (1990$).
Model learning vs input cost paths	Released GCAM takes future costs as inputs; learning curves only help choose them.

Symbol	Meaning	Where
p, P	price	2.2–2.5
F(p)	excess demand	2.2
γ, β	logit exponent (relative, absolute)	2.3
α, λ	share weight; land profit scaler	2.3, 2.6
VF	value factor of wind/solar	2.4
I, N, L	fuel intensity, non-fuel cost, load factor	2.5
W, V, S	wage rate, time-value multiplier, speed	2.5
π, ρ	land profit, land logit exponent	2.6
MAC	marginal abatement cost curve	2.8

The factory tour (memory route)

Walk the factory in order. At each stop, say the component, its rule, and one number.

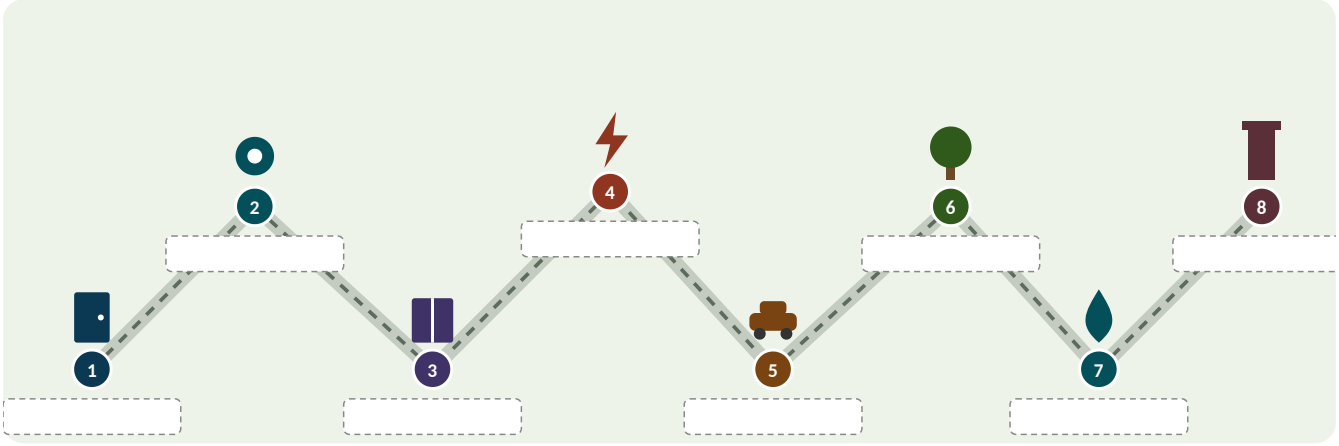


The route. Eight stops in teaching order.

1. **1. Goods-in dock** — Socioeconomic drivers
2. **2. Auction hall** — Markets, equilibrium and solving
3. **3. Chooser's desk** — Choice: the logit
4. **4. Power hall** — Energy supply
5. **5. Demand floor** — Energy demand
6. **6. Farm yard** — Land, food and bioenergy
7. **7. Water tower** — Water
8. **8. Chimney & control room** — Emissions, climate and policy: the system together

TRY IT NOW

Walk-through drill (5 min). Close the handbook. Say the eight stops aloud in order. At each, say one equation and one number (e.g. “Chooser's desk: share = $w \cdot c^{\gamma}$ over the sum; −3 for electricity”). Then fill the blank route below.



Blank route. Write the component under each stop from memory.

Quiz

Twenty questions: warm-up (6), core (10), challenge (4). Answers and routing at the end.

Warm-up

1. GCAM runs in time steps of
   A. 1 year
   B. 10 years
   C. 5 years
   D. 25 years

2. In GCAM's classic setup, regional GDP is
   A. solved by the carbon market
   B. fixed at its 2021 value
   C. an output of the energy model
   D. an input from a socioeconomic scenario

3. Which method is GCAM's main solver?
   A. Broyden's method
   B. Simplex
   C. Gradient descent
   D. Monte Carlo

4. Share weights in a logit are
   A. subsidies
   B. emission factors
   C. calibrated preference multipliers
   D. discount rates

5. How many water basins does GCAM use (common assumptions page)?
   A. 32
   B. 396
   C. 1,000
   D. 235

6. GCAM passes information to Hector as
   A. temperatures
   B. emissions
   C. prices
   D. land areas

Core

7. Costs \$60, \$70, \$80, $\gamma = -4$, equal weights. Roughly what share does the \$80 option get?
   A. 0%
   B. 54%
   C. 33%
   D. 17%

8. Making γ more negative (−3 → −10) makes the market
    A. more winner-take-all
    B. more evenly shared
    C. unchanged
    D. unsolvable
9. Why does solar's effective cost in GCAM rise as its share grows?
    A. learning stops
    B. panels wear out
    C. the value factor falls
    D. a backup turbine is added (current default)
10. Income doubles, income elasticity 0.6, prices unchanged. Demand rises by about
    A. 20%
    B. 60%
    C. 52%
    D. 100%
11. In transport, why can a pricier but faster mode win as incomes rise?
    A. the value of time rises with the wage
    B. subsidies
    C. load factors fall
    D. fuel gets cheaper
12. Land is allocated in GCAM by a logit on
    A. cost per tonne
    B. yield only
    C. carbon density only
    D. profit per unit area
13. A carbon price on land in GCAM works as
    A. a tax on farming
    B. a ban on clearing
    C. a payment for holding carbon stock
    D. a yield boost
14. Irrigation water in GCAM is charged
    A. the full basin price
    B. nothing
    C. the desalination price
    D. 5% of the general water price
15. A $100/t CO2 price expressed per tonne of carbon is about
    A. $27
    B. $367
    C. $100
    D. $3,667

16. Which statement about an emissions cap in GCAM is true?
    A. GCAM solves for the carbon price that meets it
    B. GCAM ignores caps
    C. caps are only for land
    D. caps need perfect foresight

Challenge

17. Draw it: sketch the interaction map of water (module 2.7) with at least three arrows in and two out, labelled.
18. A biofuel mandate raises bioenergy demand. Trace the effect on food prices through three components, naming the mechanism at each step.
19. Explain why GCAM can build coal plants that a later policy strands, while an optimisation model with perfect foresight would not.
20. A teammate doubles a share weight to make EVs win. Give two reasons this needs care when you report results.

If you missed…	Reread
1, 2	§2.1
3, 16, 19	§2.2 and §2.8
4, 7, 8, 20	§2.3
9	§2.4
10, 11	§2.5
12, 13, 18	§2.6
5, 14, 17	§2.7
6, 15	§2.8

Consolidate

BUILD FROM MEMORY

Build the smallest working example (15 min). Without looking, write a 10-line function that shares a market among three technologies with a relative logit, then calibrate share weights so that costs (60, 70, 80) reproduce observed shares (0.3, 0.5, 0.2) with $\gamma = -4$. **Self-check:** the gas weight relative to coal should be 3.088.

TRY IT NOW

Teach it to a 14-year-old (10 min). Explain how a biofuel mandate can raise bread prices. Banned words: *logit*, *elasticity*, *equilibrium*, *exogenous*, *marginal*.

Cheat sheet

2.1 Drivers

GDP = P × income; CO2 = P·(GDP/P)·(E/GDP)·(CO2/E). 32 regions; 2025–2100 in 5-yr steps; base year 2021. SSP2 2100: 9 bn people, $33k/person.

Goods-in dock: crates of people and banknotes.

2.2 Markets

F(p) = D − S = 0 for all markets; Broyden on log prices; tolerance ~0.001; recursive (no foresight). Lab: p 100 → 127 with $50 tax.

Auction hall: one log-scale megaphone.

2.3 Logit

$s = \alpha\, p^{\gamma} / \Sigma\, \alpha\, p^{\gamma}$; γ −3 electricity, −10 within coal; α calibrated. $60/70/80, γ −4 → 54/29/17%.

Chooser's desk: funnels and a γ dial.

2.4 Supply

C = non-energy + fuel + carbon − co-product; Q = Qmax $P^b/(Pm^b+P^b)$; PLCOE = LCOE/VF; PV VF = (0.972 − 1.443 s)/0.903.

Power hall: brick, flame, cloud stickers.

2.5 Demand

$D_t = D_{t-1}(\text{Y ratio})^{\alpha}(\text{P ratio})^{\beta}$; transport $P = (P_f\, I + N)/L + W{\cdot}V/S$. $2^{0.6} = 1.516$.

Demand floor: wristwatch price tags.

2.6 Land

$s = (\lambda\pi)^{\rho}/\Sigma(\lambda\pi)^{\rho}$; π = (p − cost) × yield; carbon price pays for carbon stock; forests regrow over 30–100 yr. 100 EJ at 10 t/ha ≈ 556 Mha.

Farm yard: wheat vs grass arm-wrestle.

2.7 Water

235 basins; renewable → groundwater → desal; irrigation pays 5%; municipal 0.37 / −0.33; RO 2.75 kWh/m³.

Water tower: three taps, farmer's voucher.

2.8 Emissions/climate

E = A·F·(1 − MAC); Hector v3.2 one-way; policies: fixedTax, cap, target, RPS; $/t C = $/t CO2 × 44/12.

Chimney: lighthouse thermometer; welded levers.

Tools for Go 2

Tool	Version / status (Oct 2026)	Install	Use in Go 2
GCAM	v9.1 (2 Jun 2026); docs describe v8.2	release package from github.com/JGCRI/gcam-core; Java	reference and policy runs (labs 7–10)
gcamreader (Python)	1.5.0 (4 Jul 2026); Python ≥ 3.11 + Java	`pip install gcamreader`	`gcam/read_gcam.py`
rgcam (R)	active, no release tag	`remotes::install_github('JGCRI/rgcam')`	labs 7–12, `gcam/read_gcam.R`
rmap (R)	active	`devtools::install_github('JGCRI/rmap')`	maps of regions and basins (lab 9)
rchart (R)	active	`devtools::install_github('JGCRI/rchart')`	standard comparison charts
hector (R)	v3.2.0 (Zenodo 2024)	`remotes::install_github('jgcri/hector')`	lab 2; compare with toy climate
FaIR (Python)	2.2.5 (15 Oct 2026)	`pip install fair`	alternative simple climate model
DICE-2023 (Python)	v0.1 beta (Ivo Welch)	download, `python3 dice.py nordhaus-base.conf`	cost-benefit contrast
MESSAGEix	message-ix 3.11.1; needs GAMS	`pip install message_ix[tutorial]`	Westeros: optimisation contrast
Archived / stale	gcamextractor, gcamland archived 8 May 2026; pygcam last release 2022	—	mention only; do not build labs on them

Versions checked on 23 October 2026 from the project pages (GitHub, PyPI, Zenodo). The GCAM documentation pages describe v8.2; the latest release is v9.1, which adds the KLEAM macro module (v9.0), updated non-CO2 MAC curves (v8.9), small modular reactors and updated techno-economic data (v8.8) and Moirai land data (v8.10).

Answer key

Try it now

1. §2.1: 1.2 × 1.5 = 1.8; × 0.7 = 1.26; × 0.8 = 1.008. CO2 rises by about 0.8%.
2. §2.2: No tax: 1000 − 2p = 6p → p = 125. With tax: 1000 − 2p = 6p − 270 → p = 158.75. Price rises 33.75 of the 45, so buyers now carry most (75%).
3. §2.3: γ = −2: 50^{-2} = 0.0004, 100^{-2} = 0.0001 → 0.0004/0.0005 = 80%. γ = −6: 2^{6} = 64 → 64/65 = 98.5%.
4. §2.4: VF(0) = 0.972/0.903 = 1.076; VF(0.3) = (0.972 − 0.433)/0.903 = 0.597. Effective cost rises by 1.076/0.597 = 1.80 times.
5. §2.5: Car: 0.30 + 10/60 = 0.467 $/km. Rail: 0.12 + 10/30 = 0.453 $/km. Rail is (just) cheaper.
6. §2.6: 30 × 10^{9} ÷ (12 × 18 = 216 GJ/ha) = 138.9 million ha.
7. §2.7: 10^{9} m³ × 2.75 kWh = 2.75 × 10^{9} kWh = 2.75 TWh.
8. §2.8: $50/t CO2 × 44/12 = $183/t C. A cheaper clean option means the cap is met more cheaply, so the solved carbon price falls.

Quiz

1. C — five-year steps 2025–2100 (§2.1).
2. D — population and GDP come from the SSP inputs; the macro module can adjust GDP (§2.1). C is the classic misconception.
3. A — Broyden's method on log prices (§2.2).
4. C — preference multipliers calibrated to history; not prices (§2.3; A is the classic misconception).
5. D — 235 (§2.7; 233 appears in the overview text, 396 is land subregions).
6. B — emissions; one-way coupling (§2.8).
7. D — 17.0% (§2.3 worked example); B is the coal share.
8. A — more negative → buyers chase the cheapest harder (§2.3, Figure 2.6).
9. C — value factor falls with share (§2.4); D was the legacy method before v7.3.
10. C — $2^{0.6}$ = 1.516 (§2.5); B wrongly treats the elasticity as linear.
11. A — time cost W·V/S grows with income (§2.5).
12. D — profit per hectare, positive exponent (§2.6).
13. C — subsidy for carbon stock (§2.6).
14. D — 5% multiplier (§2.7).
15. B — 100 × 44/12 = 366.7 (§2.8); A divides instead of multiplying.
16. A — the cap and the price are duals (§2.8).
17. Arrows in: crop production (land), cooling and mining needs (energy supply), income (drivers), runoff (Xanthos). Out: irrigation cost (land), cooling water price (energy), energy-for-water demand (§2.7, Figure 2.17).
18. Energy supply demands more biomass → energy-crop profit rises (land profit equation) → the land logit shifts cropland to energy crops → less land for food at the margin → food supply falls relative to demand → food prices rise (§2.6, loop 1).
19. GCAM is recursive: each period decides with that period's prices and no knowledge of future policy, so capital built in 2030 can be stranded by a 2040 price. A perfect-foresight optimiser sees the policy coming and avoids the plant (§2.2, §2.8).
20. Share weights are not costs, so the result reflects an assumed preference rather than economics; they also break the base-year calibration logic and are easy to hide. Report them explicitly, justify them with evidence (surveys, policy mandates), and test sensitivity (§2.3 trap).

Revision plan

When	Do this	✓
Today	Walk the factory route once; run m23_logit and m28_system	☐
Day 1	Redo the eight simple calculations without notes	☐
Day 3	Quiz Q1–Q16; reread routed sections	☐
Day 7	Draw the whole-system map from memory; compare with Figure 2.19	☐
Day 14	Trace two feedback loops in a real GCAM run (labs 9–10)	☐
Before Go 3	Challenge questions 17–20 in writing	☐

Going further

- GCAM v8.2 documentation, table of contents: https://jgcri.github.io/gcam-doc/toc.html; start with Economic Choice, Solution and Computation, Land, Water Supply and Policies.
- GCAM releases: github.com/JGCRI/gcam-core/releases (v9.1, 2 Jun 2026). Change proposals (CMP 394 intermittency; CMP 411 KLEAM) are linked from the Updates page.
- PNNL Community video tutorials (install and run GCAM, gcamdata, Hector): gcims.pnnl.gov/community.
- MESSAGEix tutorials (Westeros): docs.messageix.org.
- Calvin et al. (2019), GCAM v5.1: representing the linkages between energy, water, land, climate, and economic systems, *Geoscientific Model Development* 12, 677–698 (the model description paper; check the current version notes).

HONESTY NOTE

GCAM facts are from the GCAM v8.2 documentation pages as cited (opened 23 October 2026); where the docs contradict themselves (base year, basins, land units, protected land) the handbook says so. Parameter values not on the doc pages (plant costs, land exponents, building coefficients, Hector's climate sensitivity) are deliberately not given. All toy-model numbers (markets, logit examples, LCOE inputs, mode speeds, land prices, water tiers, the one-region system) are **illustrative**, written for teaching and computed by the scripts in the Go 2 kit. The toy climate model uses rounded textbook parameters and is not Hector. The GCAM reader scripts follow the documented APIs but were not run against a live GCAM database.